XML format for RBA models, *rbaml* version 2

Oliver Bodeit, Stephan Fischer, Anne Goelzer

February 13, 2026

Contents

1 Changes in new RBApy releases **4**
1.1 rbaml version 2 4

2 Introduction **5**
2.1 Overview of an RBA model 5
2.2 Overview of the document 5
2.3 Conventions 7
2.3.1 Naming conventions in XML and RBApy 7
2.3.2 Boolean attributes 8
2.3.3 Variables for user-defined functions 8
2.3.4 Units 8

3 metabolism.xml **10**
3.1 Rationale 10
3.2 RBAMetabolism 10
3.3 Species 11
3.4 Reaction 11
3.5 SpeciesReference 11
3.6 Examples 12

4 proteins.xml, rnas.xml, dna.xml and other_macromolecules.xml **13**
4.1 Rationale 13
4.2 RBAMacromolecules 14
4.3 Component 14
4.4 Macromolecule 15
4.5 ComponentReference 15
4.6 Examples 15

5 compartments.xml **18**
5.1 Rationale 18
5.2 RBACompartments 19
5.3 Compartment 20
5.4 CompartmentComposition 20
5.5 ConstituentReference 20

6 enzymes.xml **21**
6.1 Rationale 21
6.2 RBAEnzymes 21
6.3 Enzyme 21
6.4 Examples 22

7 processes.xml **24**
7.1 Rationale 24

7.2 RBAProcesses . . . 25
7.3 Process . . . 26
7.4 Machinery . . . 27
7.5 MachineryComposition . . . 27
7.6 Processings . . . 27
7.7 Processing . . . 27
7.8 ProcessingMap . . . 28
7.9 ConstantProcessing . . . 29
7.10 ComponentProcessing . . . 29
7.11 Examples . . . 30

8 targets.xml **33**
8.1 Rationale . . . 33
8.2 RBATargets . . . 33
8.3 TargetSpecies . . . 34
8.4 TargetReaction . . . 34
8.5 Examples . . . 35

9 parameters.xml **37**
9.1 Rationale . . . 37
9.2 RBAParameters . . . 38
9.3 Function . . . 38
9.4 Parameter . . . 40
9.5 Aggregate . . . 40
9.6 FunctionReference . . . 41
9.7 AggregateReference . . . 41
9.8 Examples . . . 42

10 custom_constraints.xml **46**
10.1 RBACustomConstraints . . . 47
10.2 Constraint . . . 47
10.3 ConstraintDefinition . . . 47
10.4 VariableReference . . . 48
10.5 ParameterReference . . . 48
10.6 Examples . . . 49

11 Mean model **50**

1 Changes in new RBApy releases

1.1 rbaml version 2

- Add file *other_macromolecules.xml* holding instances of **Macromolecule**, other than **DNA**, **RNA** or **protein**.
- Add *half_life* attribute to **Macromolecule**, representing half life time/decay.
- Add other feasible function types to **Function** (quadratic, logistic, (centered) Arrhenius ...).
- Enable competetive inhibition function type to have two independent variables (substrate and inhibitor).
- Enable addition aggregates in **Aggregate**.
- Enable nested aggregates by allowing **Aggregate** class to reference other aggregates. Introduced **AggregateReference** class.
- add *input_fraction* attribute to **Processing** class.
- Define **RBACustomConstraints** allowing to impose custom constraints onto *RBA*-model and to store them in *custom_constraints.xml*.
- Remove **RBADensity** and replaced with **RBACompartments**.
- Move definition of compartments from **RBAMetabolism** to **RBACompartments**.
- Generate a **mean** RBA model pooling all isoenzymes for a given reaction.
- Add one configuration file called *model_file_index.in* to select a xml file for compartments, metabolism, enzymes, proteins, rnas, dna, other_macromolecules, processes, targets, parameters, custom_constraints and medium.

Older models (RBApy < 3.0) can be updated with a script, included in the release.

2 Introduction

2.1 Overview of an RBA model

In Figure 1, we summarize the systemic cell description underlying to the RBA method and the mathematical relationships defining the interactions and allocation of resources between the cellular processes. All these relationships take the form of linear growth-rate dependent equalities and inequalities: for cells growing in exponential phase at a rate μ,

(I) the metabolic network has to produce all metabolic precursors necessary for biomass production (equalities C_1 in green);

(II) the capacity of all molecular machines must be sufficient to ensure their function, i.e. to catalyze chemical conversions at a sufficient rate (inequalities C_2 in blue for the enzymes and transporters, in yellow for the molecular machines of macromolecular processes);

(III) the intracellular density of compartments and the occupancy of membranes are limited (inequalities C_3 in red);

(IV) mass conservation is satisfied for all molecule types (equalities C_1 in green).

Taken together, the equalities and inequalities define, at a given rate μ, a feasibility linear programming (LP) problem that can be solved efficiently. Parsimonious resource allocation between cellular processes is modelled mathematically by optimizing the maximal cell growth, and computed by solving a series of such LP feasibility problems for different growth rate values. For a given medium, an RBA optimization problem predicts the maximal possible growth rate, the corresponding reaction fluxes, the abundances of molecular machines and the relative size of cellular compartments within the cell. Consequently, generating an RBA model requires information for formalizing constraints C_1, C_2 and C_3 (Figure 1), and in particular: (i) the localization and the composition of the molecular machines, (ii) the molecules that are consumed and released by the molecular machines for functioning; (iii) the efficiencies of molecular machines, i.e. the rates of the process per amount of the catalyzing molecular machine; (iv) parameters such as the maximal density of each compartment.

2.2 Overview of the document

In this document we present the XML structures used to define a RBA model. A complete RBA model is composed of the following files:

- metabolism.xml (definition of metabolic species and metabolic reactions).
- proteins.xml (definition of proteins).
- rnas.xml (definition of RNAs).

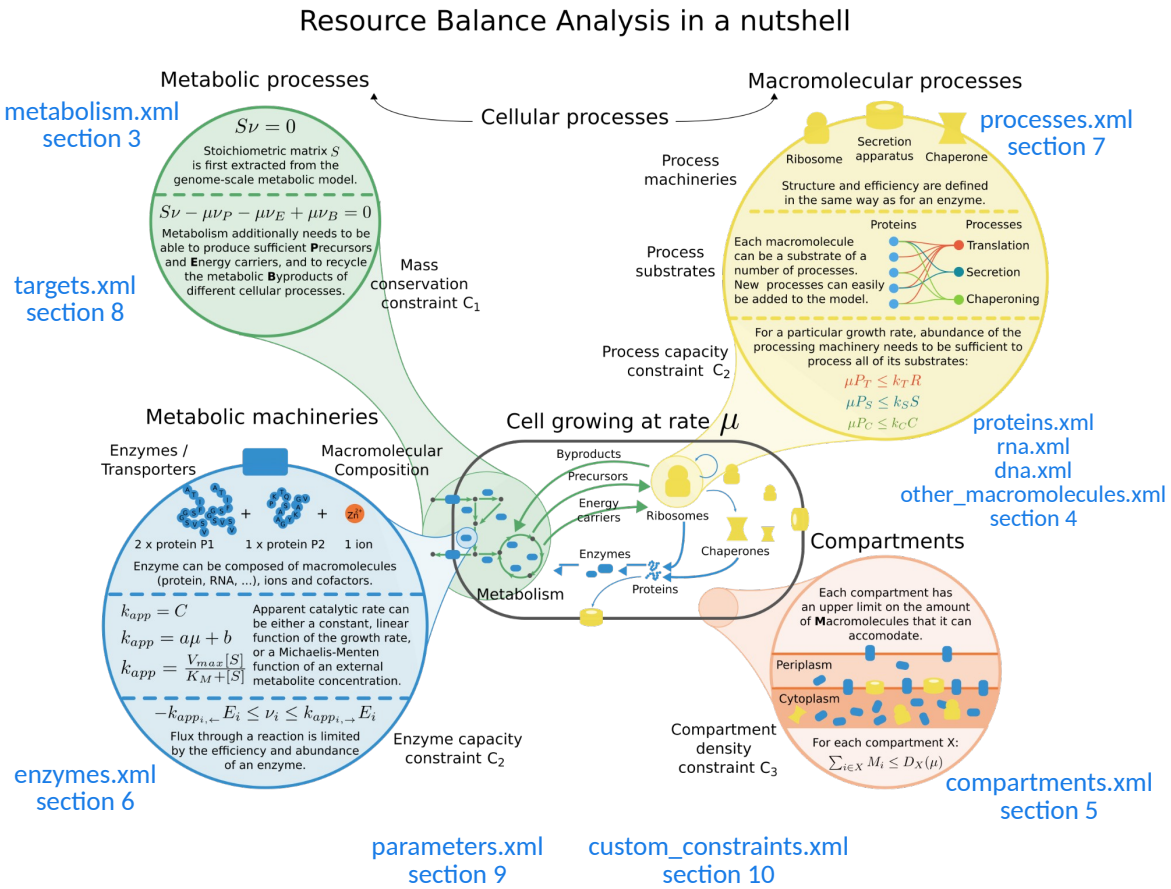

Figure 1: Overview of the files used in the XML model. Files were designed according to the 3 central constraints of RBA.

- dna.xml (definition of DNA).
- other_macromolecules.xml (definition of any other macromolecule).
- enzymes.xml (definition of enzymatic machineries catalyzing metabolic reactions).
- processes.xml (definition of cell processes used to produce macromolecules).
- compartments.xml (definition of cellular compartments and their associated constraints).
- targets.xml (definition of production or degradation constraints necessary for growth and maintenance).
- parameters.xml (definition of user-defined functions and parameters).
- custom_constraints.xml (definition of user-defined constraints).

A valid RBA model must contain all these files with these exact names in the same directory (called *model*) to be recognized by the RBApy parser. We provide also one configuration file called *model_file_index.in* to select one xml file for compartments, metabolism, enzymes, proteins, rnas, dna, other_macromolecules, processes, targets, parameters, custom_constraints and medium.

We dedicate one section of this document for every file. Each section starts with a subsection containing a brief description about the relevance of the node in the RBA model. The following subsections are more technical: we present the nodes that compose the XML structure and show a class diagram with the nodes' attributes and the children nodes that they may/must contain. The last subsection presents examples from real models and from a minimal model (see XML tutorial for RBApy).

2.3 Conventions

Before we start looking into files, we briefly review the conventions we used for the RBA format.

2.3.1 Naming conventions in XML and RBApy

We strongly advise **not** to modify XML files directly. Instead, RBApy can be used to access the XML files and modify them through scripts. Writing scripts allows you to reproduce all the modifications you have made automatically. For example, if you need to regenerate the basic XML model with preRBA, you will be able to re-apply all the modifications that you had done on your orginal model.

All XML elements can be accessed with the RBApy package. For example, the XML element Reaction can be accessed through python objects of class Reaction. This mirrorring scheme (every XML element has a corresponding python class) is inspired by the scheme used in SBML and libSBML. However, as in libSBML, the naming conventions may differ between python and XML:

- XML elements and python classes both follow `ThisConvention` (e.g. `SpeciesReference`).
- XML attributes follow `thisConvention` (e.g. `boundaryCondition`), while python attributes follow `this_convention` (e.g. `boundary_condition`).
- In XML, list elements follow the convention `listOfThings` when they are seen as an instance/subelement (e.g. `RBAMetabolism` contains one instance of `ListOfSpecies` called `listOfSpecies`). In python, the `listOf` prefix is dropped and lower case is used (e.g. `RBAMetabolism` contains one instance of `ListOfSpecies` called `species`).

Throughout this document, XML conventions are used. Please keep in mind that the convention for attributes/instances is different when using the python package.

2.3.2 Boolean attributes

A boolean attribute evaluates to true if it is `"1"` or `"true"` (case does not matter). In all other cases it evaluates to false.

2.3.3 Variables for user-defined functions

The default variable for functions is the growth rate. It can also be explicitly defined as by the string `growth_rate`. Alternatively, a function can take as an input the value of an environmental variable (defined in the file *medium.tsv*) such as the *external* concentration of a metabolite (e.g. for transport functions) or the temperature. A metabolite identifier is expected to look like `met_c`, where `met` is the name of the metabolite and `c` its compartment. Note that in the current version of RBApy, every time a function based on `met_c` is evaluated, the compartment suffix is ignored and the extracellular concentration of the metabolite is used *no matter where the transport takes place*. Typically, glucose import rates from the periplasm to the cytosol will be based on the concentration of extracellular glucose.

2.3.4 Units

In the current version of RBApy, units are implicitly defined! It is left to the user to make sure that the model is consistent. For example, density constraints (C_3) rely on a weight unit that is used in the compartment file (compartments.xml), the macromolecule files (proteins.xml, rnas.xml, dna.xml), and the global constraints imposing that the densities of cellular compartments must sum to the total density of the cell (custom_constraints.xml). You need to make sure that you use the same unit in all files! In the density example, our group usually uses amino acids as a weight unit. The compartment and custom_constraints files set an upper limit in the number of amino acids that can be contained in the cell. The protein file defines the weight of all amino acid residues to be 1. For RNAs and DNA, we convert the weight of nucleotides to amino acid equivalents. For example, the weight of an Adenosine residue is 2.9036 (a nucleotide is heavier than an amino acid).

The unit of time for metabolic fluxes, efficiencies of molecular machines is also implicit ! Usually, we used 1/hour for microorganisms, 1/day for multicellular organisms such as plants.

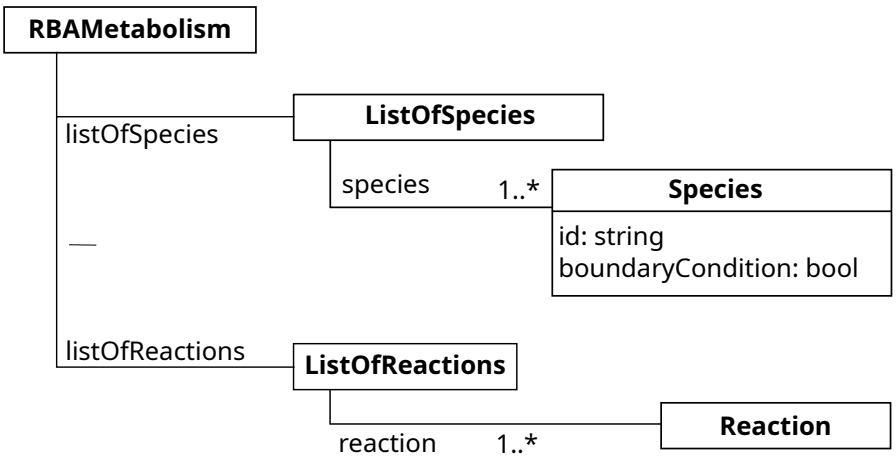

Figure 2: XML structure of metabolism document.

3 metabolism.xml

The metabolism file is strongly inspired by SBML. More precisely, it can be seen as a subpart of an SBML file. It is used to define metabolic species and reactions.

3.1 Rationale

metabolism.xml contains the most basic bricks of an RBA model. In our effort to define a minimal structure that contains an RBA model, we decided to start with an SBML structure and strip it down to elements that are essential to RBA.

metabolism.xml defines the structure of the metabolic network: simple chemical species (metabolites) that flow between compartments through transport reactions or transformed into other simple chemical species that will be available as building blocks for more complex molecules.

The description is entirely static: input fluxes are defined through the medium and parameters.xml, output fluxes are defined by targets.xml, the dynamics of internal fluxes is defined in enzymes.xml.

3.2 RBAMetabolism

The outermost part of the metabolism file is an instance of class **RBAMetabolism**, shown in Figure 2.

Currently, **RBAMetabolism** has no simple attributes. It includes exactly one instance of each **ListOf** container class. All **ListOf** classes do not have own attributes, they are merely used to organize a list of instances from another class. This organization was inspired by SBML.

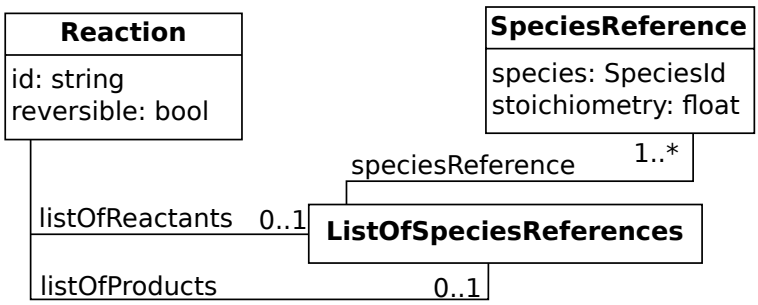

Figure 3: Class storing metabolic reactions.

3.3 Species

The **Species** class is used to define *metabolic* species.

The *id* attribute The **id** attribute is a string defining the identifier of a metabolite.

The *boundaryCondition* attribute The **boundaryCondition** attribute is a boolean. If the attribute is set to true, the metabolite is considered to be at a constant concentration. In other words, it is not affected by reactions. This is typical for metabolites in the external medium.

3.4 Reaction

The **Reaction** class is used to define metabolic reactions (Fig. 3). Reactants and products are defined using a **ListOfSpeciesReferences**.

The *id* attribute The **id** attribute is a string defining the identifier of a reaction.

The *reversible* attribute The **reversible** attribute is a boolean. If the attribute is set to true, the reaction can occur in both directions. If the attribute is set to false, only the forward reaction can occur.

3.5 SpeciesReference

The **SpeciesReference** class is used to refer to a metabolic **Species** and associate with it a stoichiometry (Fig. 3).

The *species* attribute The **species** attribute must match the identifier of a **Species**.

The *stoichiometry* attribute The **stoichiometry** is a positive real number. It repensents the stoichiometry of a **Species** in a given context (typically a **Reaction**).

```xml
<RBAMetabolism>
  <listOfSpecies>
    <species id="M_carbon_source_e" boundaryCondition="true"/>
    <species id="M_carbon_source_c" boundaryCondition="false"/>
    <species id="M_protein_component_precursor_c" boundaryCondition="false"/>
    <species id="M_biomass_c" boundaryCondition="false"/>
  </listOfSpecies>
  <listOfReactions>
    <reaction id="R_transport" reversible="false">
      <listOfReactants>
        <speciesReference species="M_carbon_source_e" stoichiometry="1.0"/>
      </listOfReactants>
      <listOfProducts>
        <speciesReference species="M_carbon_source_c" stoichiometry="1.0"/>
      </listOfProducts>
    </reaction>
    <reaction id="R_protein_component_precursor" reversible="false">
      <listOfReactants>
        <speciesReference species="M_carbon_source_c" stoichiometry="1.0"/>
      </listOfReactants>
      <listOfProducts>
        <speciesReference species="M_protein_component_precursor_c" stoichiometry="1.0"/>
      </listOfProducts>
    </reaction>
    <reaction id="R_transport" reversible="false">
      <listOfReactants>
        <speciesReference species="M_carbon_source_c" stoichiometry="1.0"/>
      </listOfReactants>
      <listOfProducts>
        <speciesReference species="M_biomass_c" stoichiometry="1.0"/>
      </listOfProducts>
    </reaction>
  </listOfReactions>
</RBAMetabolism>
```

Figure 4: metabolism.xml from the minimal model with 4 metabolites and 3 reactions.

The *comment* attribute The (optional) **comment** can contain any string for annotation purposes.

3.6 Examples

Figure 4 shows a very simple example with 4 metabolites and 3 reactions. In this example, we tagged `M_carbon_source_e` with `boundary_condition="true"`, implying that it is an external metabolite whose concentration is known and set through the medium in *medium.tsv*. Boundary metabolites are essential in the model, as they define input fluxes in the model.

Note that the description of the metabolic network ends with the protein precursor. Proteins *should not* be defined in metabolism.xml. Their composition is described in proteins.xml, while their assembly is described in processes.xml.

4 proteins.xml, rnas.xml, dna.xml and other_macromolecules.xml

All these files are base on the classes **RBAProteins**, **RBARnas**, **RBADna** and **RBA-Macromolecules**. The first three inherit all of their functionality from **RBAMacromolecules**, only differing by the specific xml-tags *RBAProteins*, *RBARnas*, *RBADna* and *RBAMacromolecules*.

4.1 Rationale

In RBA, basic molecules are seen either as metabolites or macromolecules. In the previous versions of RBApy, macromolecules encompassed only the polymer molecules proteins, RNAs and DNA. We now introduce a general class *other_macromolecules* to represent any molecular species (e.g. polymers like fibers, lipids or even small molecules) as *macromolecule*, with polymer-like properties in their assembly or degradation processes.

The formal distinction between macromolecules and other molecules is that macromolecules can be defined as an ensemble of other, simpler molecules or molecule residues. Elements of a macromolecule family (e.g. proteins, RNAs, DNA) are based on the same subset of component molecules (amino acid residues, vitamins, ions for proteins), share common assembly processes, but differ in the stoichiometry of components they are built of.

For example, a protein is often described as a sequence of letters, say "MAGLKYAAALK", where every letter implicitly represents a component (an amino acid residue). It may also contain post-translational modifications, such as a phosphorylation on the tyrosine Y. The RBA format starts by listing the components that are common to all proteins: all amino acid residues (A, C, .., Y), vitamins, ions and other cofactors. The second part of the RBA format lists all proteins broken down as an ensemble of components. For example, the protein above would be described as {(A:4), (G:1), (K:2), (L:2), (M:1), (Y:1), (Phospho:1)}, where Phospho represents the tyrosine phosphorylation (in this example, we pooled all possible phosphorylations into a single component).

Note that macromolecule components are *not* metabolites. How they are built from metabolites is part of the assembly process defined in processes.xml. They may share identical identifiers as metabolites, but we advise using different identifiers for clarity. For example, an amino acid residue is *not* an amino acid: it is obtained by loading a charged tRNA onto a nascent protein with the appropriate energetic molecules, while releasing several byproducts, including an uncharged tRNA. In a sense, a component can be seen as an *atomic assembly action*, rather than a submolecule.

The macromolecule files contain static descriptions of the macromolecules. How they are assembled from metabolites and the machines needed to assemble them are defined in processes.xml.

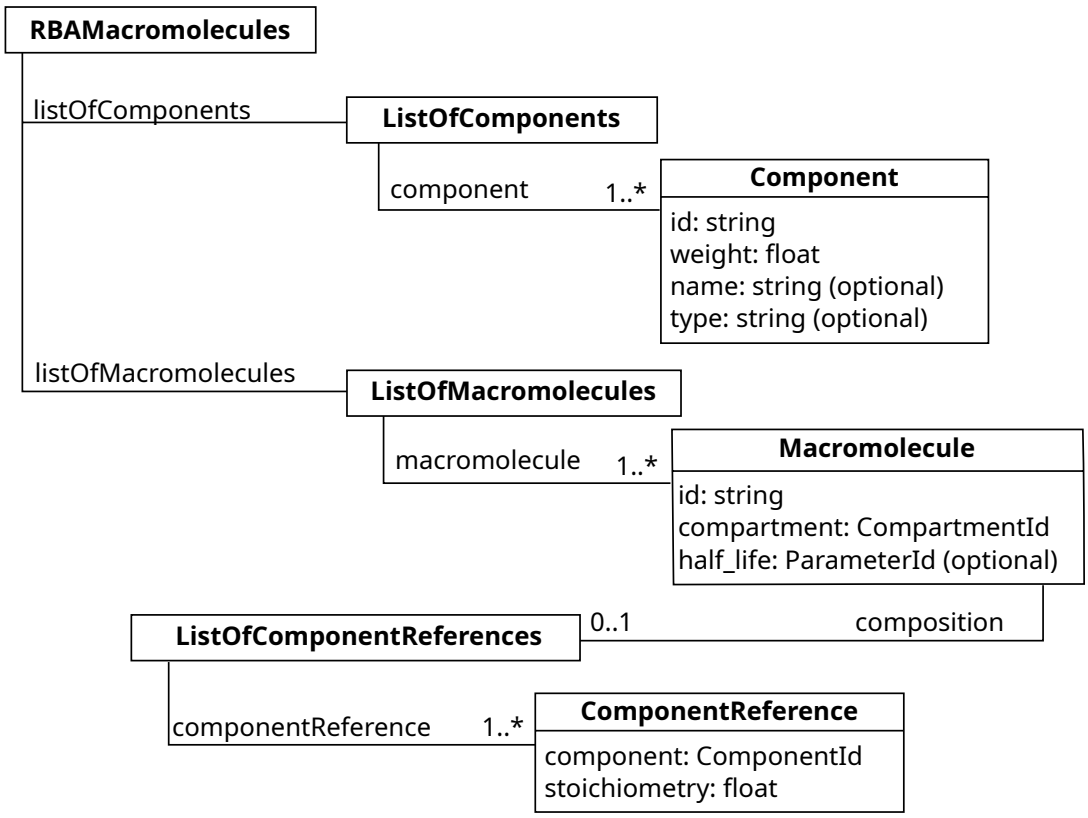

Figure 5: XML structure of macromolecule document.

4.2 RBAMacromolecules

The outermost part of the protein, RNA, DNA and other_macromolecules files is an instance of class **RBAMacromolecules**, shown in Figure 5.

RBAMacromolecules has no simple attributes. It contains exactly one instance of **ListOfComponents** and **ListOfMacromolecules**.

4.3 Component

The **Component** class is used to define the components of a **Macromolecule** (Fig. 5). For example, these are expected to be amino acids, vitamins and ions for proteins. Even if there is a strong connection between metabolic **Species** and **Component**s, they are seen as independent entities with separate identifiers. The connection between **Species** and **Component**s is established in the process file, where **ProcessingMap**s define how components are assembled from metabolites.

The *id* attribute The **id** attribute is a string defining the identifier of a component.

The *weight* attribute The **weight** attribute is a real number defining the weight of a component. This information is essential for the density constraints (C_3). The weight of

a macromolecule is defined as the sum of the weight of its components. The weight unit is unspecified, however it should be consistent with the parameters used in the density constraints.

The *name* and *type* attributes The **name** and **type** attributes are strings that provide additional information about the component. The name is a standard name of the component (*e.g.* full amino acid name). The type can be used to distinguish components if necessary (*e.g.* in amino acids, vitamins, ions).

4.4 Macromolecule

The **Macromolecule** class is used to define macromolecular species (Fig. 5). Its composition is given by a **ListOfComponentReferences**.

The *id* attribute The **id** attribute is a string defining the identifier of the macromolecule.

The *compartment* attribute The **compartment** attribute must match the identifier of a **Compartment**. It represents the compartment where the molecule is thought to be active.

The *half_life* attribute The **half_life** attribute represents the half life time of the macromolecule. It must match the identifier of a parameter (**Function** or **Aggregate**) in parameters.xml. If this attribute is not provided in the xml file, the molecule is considered to be stable ($t_{1/2} = \infty$).

4.5 ComponentReference

The **ComponentReference** class is used to refer to a **Component** and associate with it a stoichiometry (Fig. 5).

The *component* attribute The **component** attribute must match the identifier of a **Component** defined in the same **RBAMacromolecules** instance.

The *stoichiometry* attribute The **stoichiometry** is a positive real number. It represents the stoichiometry of a **Component** (typically how often it appears in a **Macromolecule**, e.g. the number of alanine residues in a protein).

4.6 Examples

The list of components and complexity of macromolecule definitions depends on the desired level of detail. In real models, we include a large list of possible components including all amino acid residues and cofactors for proteins (Fig. 6). In processes.xml,

we will define how each component is built from metabolites. In particular, a protein can only be built if all components can be assembled, including cofactors. For example, BSU29680 cannot be produced in the absence of magnesium. Note that in this model, we consider that all amino acid residues have a weight of 1, while cofactors have a weight of 0. This is a simplifying assumption that enables us to estimate the overall weight of proteins in the model. We consider that this level of granularity is sufficient to define density constraints at a satisfactory resolution.

```
<RBAProteins>
  <listOfComponents>
    <component id="A" name="" type="amino_acid" weight="1"/>
    <component id="C" name="" type="amino_acid" weight="1"/>
    <component id="D" name="" type="amino_acid" weight="1"/>
    <component id="E" name="" type="amino_acid" weight="1"/>
    ...
    <component id="Y" name="" type="amino_acid" weight="1"/>
    <component id="CHEBI:18420" name="Mg(2+)" type="cofactor" weight="0"/>
    ...
    <component id="CHEBI:597326" name="pyridoxal 5'-phosphate" type="cofactor" weight="0"/>
  </listOfComponents>
  <listOfMacromolecules>
    <macromolecule id="BSU29680_loc_Cytoplasm" compartment="Cytoplasm" half_life="default_protein_half_life">
      <composition>
        <componentReference component="T" stoichiometry="25.0"/>
        <componentReference component="Y" stoichiometry="23.0"/>
        ...
        <componentReference component="W" stoichiometry="17.0"/>
        <componentReference component="C" stoichiometry="3.0"/>
        <componentReference component="CHEBI:18420" stoichiometry="1.0"/>
      </composition>
    </macromolecule>
  </listOfMacromolecules>
</RBAProteins>
```

Figure 6: proteins.xml from the model automatically generated by RBApy for the model bacteria *B. subtilis*. Large chunks of the files were removed for brevity. The file contains two lists. The component list contains construction blocks for proteins: amino acid residues and cofactors such as ions and vitamins. The macromolecule list defines all the proteins in the system. Here we show one example of cytosolic proteins, defined as an ensemble of amino acid residues and cofactors, and with the half life time.

Proteins, RNAs and DNA are defined on the same vein. On figure Fig. 7, we give an example of the definition of a new macromolecule (e.g. a membranous phospholipid) in *other_macromolecules.xml*. In some cases, it could be useful that a metabolic species is treated as a macromolecule. We also show on Fig. 7 how it can be done on magnesium as an example.

```xml
<RBAMacromolecules>
  <listOfComponents>
    <component id="CHEBI:18420" name="Mg(2+)" type="cofactor" weight="0"/>
    <component id="C14_FA" name="C14_fatty_acid" type="fatty_acid" weight=""/>
    <component id="C18_FA" name="C18_fatty_acid" type="fatty_acid" weight="0"/>
    <component id="PE_PLH" name="PE_phospholipid_head" type="phospholipid_head" weight="0"/>
  </listOfComponents>
  <listOfMacromolecules>
    <macromolecule id="dummy_macromolecule_Mg2_loc_Cytoplasm" compartment="Cytoplasm">
      <composition>
        <componentReference component="CHEBI:18420" stoichiometry="1.0"/>
      </composition>
    </macromolecule>
    <macromolecule id="dummy_phospholipid__loc_Cell_membrane" compartment="Cell_membrane" half_life="default_membrane_lipid_half_life">
      <composition>
        <componentReference component="PE_PLH" stoichiometry="1.0"/>
        <componentReference component="C14_FA" stoichiometry="1.0"/>
        <componentReference component="C18_FA" stoichiometry="2.0"/>
      </composition>
    </macromolecule>
  </listOfMacromolecules>
</RBAMacromolecules>
```

Figure 7: other_macromolecules.xml file, specifying a phospholipid of the membrane and defining a small molecule (magnesium ion) as macromolecule.

5 compartments.xml

The compartments.xml file defines the cellular compartments, with respective sizes and (relative) composition.

5.1 Rationale

A cell can only contain a limited number of macromolecules: there is a threshold that the total density cannot exceed. Every macromolecule has a weight W_m that equals the sum of its components' weights w_c multiplied with their stoichiometry s_c:

$$W_m = \sum_{c \in [1..C_m]} (s_c w_c)$$

Where C_m is the total number of components in macromolecule m.

We assign to every macromolecule a concentration c_i. The total occupation of compartment (k) is defined as:

$$\sum_{m \in [1..M_k]} (c_m W_m)$$

where M_k is the total number of macromolecules in compartment (k). The total occupation of the cell is defined as

$$\sum_{m \in [1..M_T]} (c_m W_m)$$

where M_T is the total number of macromolecules in the cell.

By introducing the decision variable ρ_k for the occupation of compartment (k), the occupation constraint for each compartment is defined by:

$$\sum_{i \in [1..M_k]} (c_i W_i) - \rho_k = 0$$

Moreover, in order to ensure the global cellular integrity, the occupations of all cellular compartments must not exceed the total cellular density:

$$\sum_{k \in [1..N_k]} (\rho_k) = \rho_{tot}$$

where N_k is the total number of *internal* compartments in the cell.

The occupation constraints of each cellular compartment and the global occupation constraint allow a free allocation of occupation between compartments (i.e. the decision variables ρ_*).
Measured compartment occupations can be imposed by constraining the ρ_* decision variables to measured values, as it was done in previous applications of *RBA* to prokaryotes.

$$\rho_k^{min} \leq \rho_k \leq \rho_k^{max}$$

The XML format allows to specify lower and upper bounds on each compartment occupation, where it is fully constrained if $\rho_k^{min} = \rho_k^{max}$.

While the occupation accounted for so far, relates to the macromolecules which constitute cellular machinery; cellular compartments may also be composed of other (housekeeping) macromolecules (such as lipids, housekeeping protein, stored nutrients ...), leaving less *"space"* for *functional* machinery components.

$$\sum_{i \in [1..M_k]} (c_i W_i) - (1 - f_1 - \ \ldots \ - f_{C_k}) \ \rho_k = 0$$

where the f_* coefficients correspond to the fraction occupied by a given macromolecular species in the cellular compartment (k). The relative composition of compartments $(w.r.t$ different housekeeping macromolecules and their relative amount in the compartment) can be specified in the xml files, according to the class structure outlined in Figure 8.

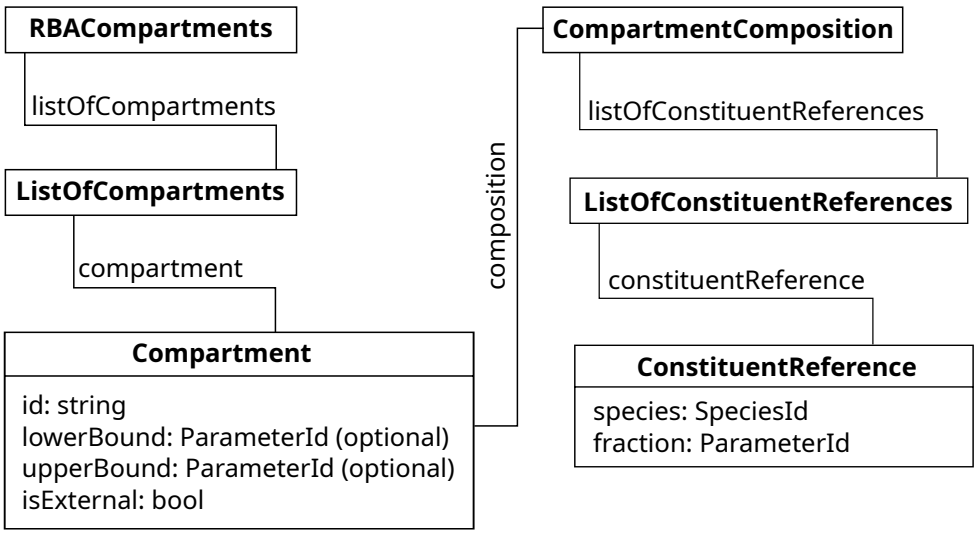

Figure 8: XML structure of compartments document.

5.2 RBACompartments

The outmost part of the compartments xml structure is an instance of the class **RBA-Compartments**. This class has n o simple attributes and only holds instance of the **ListOf** container class (holding instances of the class **Compartment**). All **ListOf** classes do not have own attributes, they are merely used to organize a list of instances from another class.

5.3 Compartment

The **Compartment** class is used to define cell compartments. It must hold information on occupation bounds of the respective compartments and may define the relative composition with the **CompartmentComposition** class.

The *id* attribute The **id** attribute is a string defining the identifier of the compartment. Every compartment should have a different id.

The *isExternal* attribute The **isExternal** attribute is a boolean indicating whether a compartment is externa to the cell (therefore not occupying "volume" in the cell).

The *lowerBound* attribute The **lowerBound** attribute defines the lower limit of the compartment occupation. It must link to a model-parameter.

The *upperBound* attribute The **upperBound** attribute defines the upper limit of the compartment occupation. It must link to a model-parameter.

If **lowerBound** and **upperBound** are defined for the compartment, the compartment occupation decision variable is constrained between the two.

5.4 CompartmentComposition

This class defines the composition of a given compartment *w.r.t* various macromoleculear housekeeping species and their relative occupation of the compartment in question.
It has no simple attributes and only holds instance of the **ListOf** container class (holding instances of the class **ConstituentReference**). All **ListOf** classes do not have own attributes, they are merely used to organize a list of instances from another class.

5.5 ConstituentReference

This class defines the fraction occupied by a given macromolecular species in the cellular compartment.

The *species* attribute The **species** attribute refers to a macromolecular species, defined in any of the files referring to **RBAMacromolecules** (*proteins.xml*, *rnas.xml*, *dna.xml* and *other_macromolecules.xml*).

The *fraction* attribute The **fraction** attribute must link to a model-parameter (with a value between 0 and 1).

6 enzymes.xml

6.1 Rationale

The enzyme file is used to define enzyme composition and their catalytic efficiency. It defines efficiency constraints in the RBA model. These constraints ensure that a reaction flux is smaller than the product of efficiency and concentration of the enzyme catalyzing the reaction.

So far, we have defined two types of basic molecules: metabolites (in metabolism.xml) and macromolecules (e.g. in proteins.xml). There is a third type of molecule in RBA models called a *molecular machine*.

Molecular machines are composed of metabolites and macromolecules. Contrary to macromolecules, who share a small pool of components and undergo complex assembly processes, molecular machines can be composed of any metabolite or macromolecule, and their assembly is described by a single reaction. Molecular machines also have a functional role within the cell: catalyzing metabolic reactions or assisting the assembly of macromolecules.

Enzymes are the simplest molecular machines in RBA models, their role is to catalyze metabolic reactions (as defined in metabolism.xml). Process machines are the other molecular machines in RBA models, their composition is defined identically to enzymes, but characterizing their role in the assembly of macromolecules, although similar to the catalysis description, is significantly more complex (as we will see in processes.xml).

The definition of enzymes contains three parts: the composition of the enzyme, the reaction the enzyme catalyzes, and the forward and backward efficiencies of the enzyme. Only one enzyme can be associated with a reaction: if a reaction may be catalyzed by several enzyme, it must be duplicated so that every enzyme can be associated with a separate reaction.

6.2 RBAEnzymes

The outermost part of the metabolism file is an instance of class **RBAEnzymes**, shown in Figure 9.

RBAEnzymes has no simple attributes. It contains excatly one instance of **ListOfEnzymes** that is used to store **Enzyme** information.

6.3 Enzyme

The **Enzyme** class is used to define enzymes (Fig. 9).

It contains a **MachineryComposition** that refers to metabolic **Species** and **Macromolecule**s composing the **Enzyme** as a **SpeciesReference**. Note that the composition can be left unspecified. In this case, the reaction associated with the enzyme is considered spontaneous. Note also that for enzymes, the **comment** attribute of the **SpeciesReference** is automatically created with the subunit number (e.g. SU_1 for the first macromolecules composing the machinery, SU_2 for the second, etc).

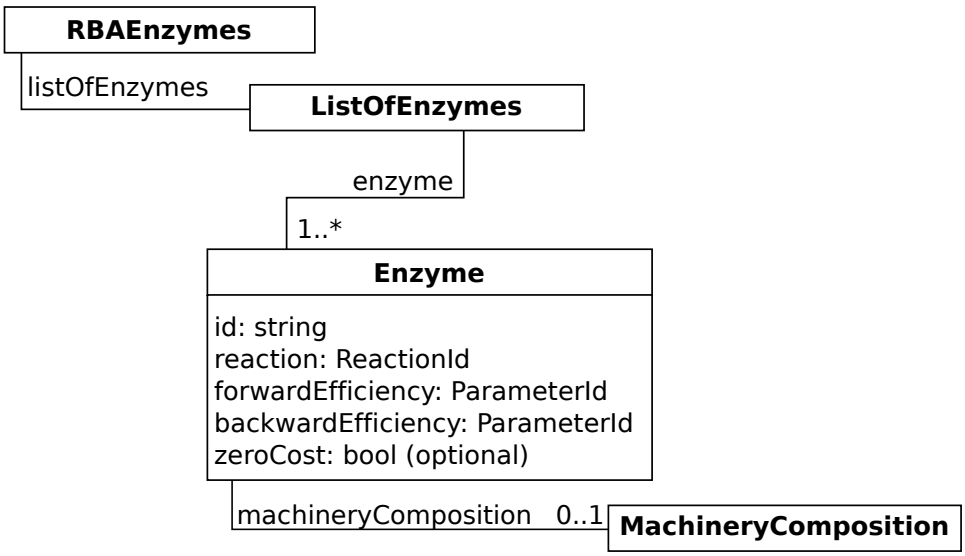

Figure 9: XML structure of enzyme document.

The *id* attribute The **id** attribute is a string defining the identifier of the enzyme.

The *reaction* attribute The **reaction** attribute must match the identifier of a metabolic **Reaction**. It represents the reaction catalyzed by the enzyme. This must be a one-to-one mapping. A **Reaction** can only have one associated **Enzyme**. If several **Enzyme**s catalyze the same **Reaction**, the **Reaction** must be duplicated.

The *forwardEfficiency* attribute The **forwardEfficiency** attribute must match the identifier of a parameter (**Function** or **Aggregate**). It represents the forward catalytic constant.

The *backwardEfficiency* attribute The **backwardEfficiency** attribute must match the identifier of a parameter (**Function** or **Aggregate**). It represents the backward catalytic constant (only applicable if reaction catalyzed by enzyme is reversible).

The *zeroCost* attribute The **zeroCost** attribute is a boolean value. If set to true, the reaction associated may occur without having to produce the enzyme. If set to false or unspecified, an efficiency constraint is created where the flux through the reaction has to be smaller than the product of enzyme efficiency and enzyme concentration.

6.4 Examples

A typical enzymes.xml contains a long list of enzymes (Fig. 10), typically one enzyme per metabolic reaction (Fig. 11). If a reaction occurs spontaneously, it is not technically necessary to associate a reaction with it, but RBApy creates an enzyme none-the-less, showing that the reaction was correctly identified as spontaneous.

```
<RBAEnzymes>
  <listOfEnzymes>
    <enzyme id="Ethi1_enzyme" reaction="Ethi1" forward_efficiency="Ethi1_enzyme_forward_efficiency" backward_efficiency="Ethi1_enzyme_backward_efficiency" zeroCost="false"/>
    <enzyme id="Ethi2_enzyme" reaction="Ethi2" forward_efficiency="Ethi2_enzyme_forward_efficiency" backward_efficiency="Ethi2_enzyme_backward_efficiency" zeroCost="false"/>
    <enzyme id="Ethi3_enzyme" reaction="Ethi3" forward_efficiency="Ethi3_enzyme_forward_efficiency" backward_efficiency="Ethi3_enzyme_backward_efficiency" zeroCost="false"/>
    <enzyme id="Ethi4_enzyme" reaction="Ethi4" forward_efficiency="Ethi4_enzyme_forward_efficiency" backward_efficiency="Ethi4_enzyme_backward_efficiency" zeroCost="false"/>
    <enzyme id="TfruLev_enzyme" reaction="TfruLev" forward_efficiency="TfruLev_efficiency" backward_efficiency="TfruLev_enzyme_backward_efficiency" zeroCost="false">
      <machineryComposition>
        <listOfReactants>
          <speciesReference species="BSU27070_loc_Cytoplasm" stoichiometry="1.0" comment="SU_1"/>
          <speciesReference species="BSU27060_loc_Cytoplasm" stoichiometry="1.0" comment="SU_2"/>
          <speciesReference species="BSU27050_loc_Cell_membrane" stoichiometry="1.0" comment="SU_3"/>
          <speciesReference species="BSU27040_loc_Cell_membrane" stoichiometry="1.0" comment="SU_4"/>
        </listOfReactants>
      </machineryComposition>
    </enzyme>
    <enzyme id="TmalP_enzyme" reaction="TmalP" forward_efficiency="TmalP_efficiency" backward_efficiency="TmalP_enzyme_backward_efficiency" zeroCost="false">
      <machineryComposition>
        <listOfReactants>
          <speciesReference species="BSU08200_loc_Cell_membrane" stoichiometry="1.0" comment="SU_1"/>
        </listOfReactants>
      </machineryComposition>
    </enzyme>
        ...
  </listOfEnzymes>
</RBAEnzymes>
```

Figure 10: enzymes.xml from a model automatically generated by RBApy for the model bacteria *B. subtilis*. Large chunks of the files were removed for brevity. The list starts with 4 enzymes with no composition, indicating that the associated reactions occur spontaneously. The last two enzymes have different compositions.

The **MachineryComposition** is identical to a metabolic reaction, except that it may contain macromolecules, not just metabolites. In the two examples, enzymes are composed of proteins, with identifiers defined in proteins.xml. The reaction could also have included byproducts by defining a `ListOfProducts`. A typical example would have been the inclusion of GTP as a reactant and GDP as a byproduct of the assembly of some enzymes, but we neglected these costs in our models.

Note that the efficiencies are *not* numerical values: they are parameter identifiers, all parameters of the model being defined in parameters.xml. Also note that `zero_cost` is a flag that is almost always set to false. Setting it to true effectively removes the enzyme, making the associated reaction spontaneous. It was included for retrocompatibility with early RBA versions, where it was used to counterbalance numerical instabilities.

```xml
<RBAEnzymes>
  <listOfEnzymes>
    <enzyme id="R_transport_enzyme" reaction="R_transport" forward_efficiency="kcat_transport" backward_efficiency="zero" zeroCost="false">
      <machineryComposition>
        <listOfReactants>
          <speciesReference species="large_protein" stoichiometry="2.0" comment="SU_1"/>
        </listOfReactants>
      </machineryComposition>
    </enzyme>
    <enzyme id="R_protein_precursor_enzyme" reaction="R_protein_component_precursor" forward_efficiency="kcat_precursor" backward_efficiency="zero" zeroCost="false">
      <machineryComposition>
        <listOfReactants>
          <speciesReference species="small_protein" stoichiometry="2.0" comment="SU_1"/>
        </listOfReactants>
      </machineryComposition>
    </enzyme>
    <enzyme id="R_biomass_enzyme" reaction="R_biomass" forward_efficiency="kcat_biomass" backward_efficiency="zero" zeroCost="false">
      <machineryComposition>
        <listOfReactants>
          <speciesReference species="small_protein" stoichiometry="1.0" comment="SU_1"/>
          <speciesReference species="large_protein" stoichiometry="1.0" comment="SU_2"/>
        </listOfReactants>
      </machineryComposition>
    </enzyme>
  </listOfEnzymes>
</RBAEnzymes>
```

Figure 11: enzymes.xml from the minimal model. We associate one enzyme with all 3 reactions defined in metabolism.xml. Enzymes are defined by using the proteins that we defined in proteins.xml.

7 processes.xml

7.1 Rationale

The process file is used to define cellular processes involved in the production or degradation of macromolecules. In proteins.xml, rnas.xml and dna.xml, we have described proteins in terms of components (amino acid residues, nucteotides, protein cofactors). processes.xml defines how components are assembled from metabolites and what machines are needed to catalyze the assembly.

A typical example of a cellular process is protein translation. Translation relies on ribosomes as a molecular machine, and participates in protein production. The metabolites used during proteins assembly include charged and uncharged-tRNAs and GTP/GDP, e.g. an alanine residue is obtained by using a charged tRNA-alanine and GTP as reactants, generating an uncharged tRNA, GDP and water as a byproduct.

Theoretically, the translation of a protein could be written down as a single reaction, listing all charged-/uncharged-tRNAs, GTP/GDP, according to the SBML format. In a model containing 1000 proteins, we would have to write 1000 reactions that follow the same template. Now, if we add other processes undergone by proteins (translation, chaperoning, translocation), we would have to add more reactions per protein to account for the fact that each process is catalyzed by the appropriate molecular machine such as an enzyme or a ribosome. For example, if all proteins need chaperones for folding, we need to add 1000 reactions for chaperoning to the pre-existing 1000 reactions for reaction synthesis. If we add translocation/transport of proteins to target locations, that's 1000 more reactions. If we update the template of one of the process, e.g. we revise the number of GTPs that are consumed per amino acid for protein synthesis, we need to rewrite all 1000 reactions describing protein synthesis, which is an extremely error-prone process.

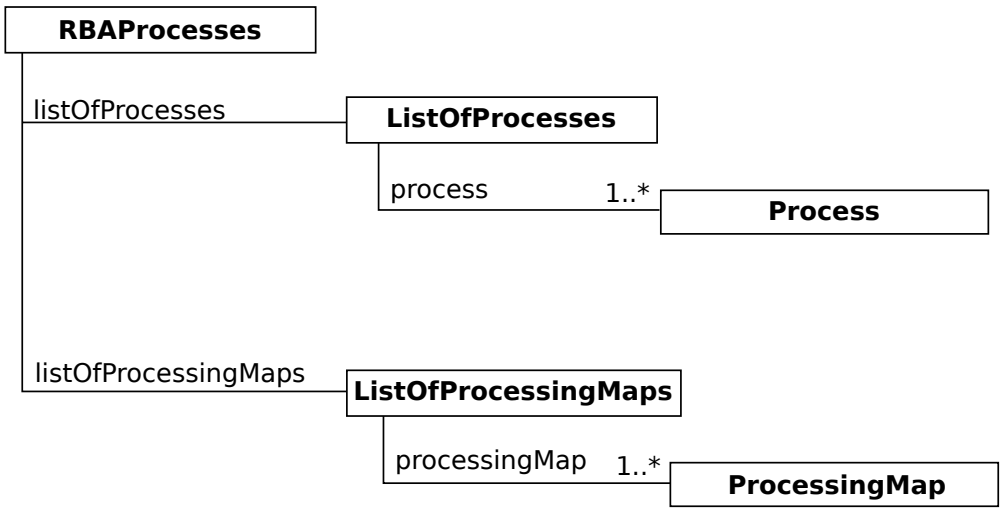

Figure 12: XML structure of process document.

RBA **Process**es can be seen as template reactions that specify how **Macromolecule**s are produced and degraded based on their **Component**s. Instead of writing one reaction per protein, we write one generic reaction per process, and proteins are seen as input of these processes. Every cellular process is described by (1) the molecular machine that catalyzes the process (e.g. the ribosome), (2) the list of macromolecules to be processed (e.g. proteins), (3) the processing costs (e.g. tRNAs and GTP consumed to assemble an amino acid), and (4) the efficiency of the molecular machine in catalyzing the process, which corresponds to the rate of the process per amount of process machinery (e.g. translation rate for ribosomes in amino acids per hour).

The processing costs are defined by a **ProcessingMap**, which explicits how every **Component** is processed, i.e. what **Species** are consumed and what **Species** are produced as byproducts of the assembly process. For example, in order to assemble an Alanine residue, a charged alanine-tRNA is consumed along with GTP, and an uncharged alanine-tRNA and GDP are generated as byproducts. **ProcessingMap**s define one reaction per **Component**, frow which the overall production/degradation reactions of **Macromolecule**s can be deduced.

Processes are flexible: **Macromolecule**s can be listed as input of several processes. For example, proteins may undergo translation, folding and translocation. The overall production/degradation reaction of a **Macromolecule** reflects all the **Process**es it traverses. When we modify a process, e.g. we update the number of GTPs consumed per amino acid, it automatically affects the production of all input macromolecules.

7.2 RBAProcesses

The outermost part of the process file is an instance of class **RBAProcesses**, shown in Figure 12.

RBAProcesses has no simple attributes. It contains exactly one instance of **ListOf-**

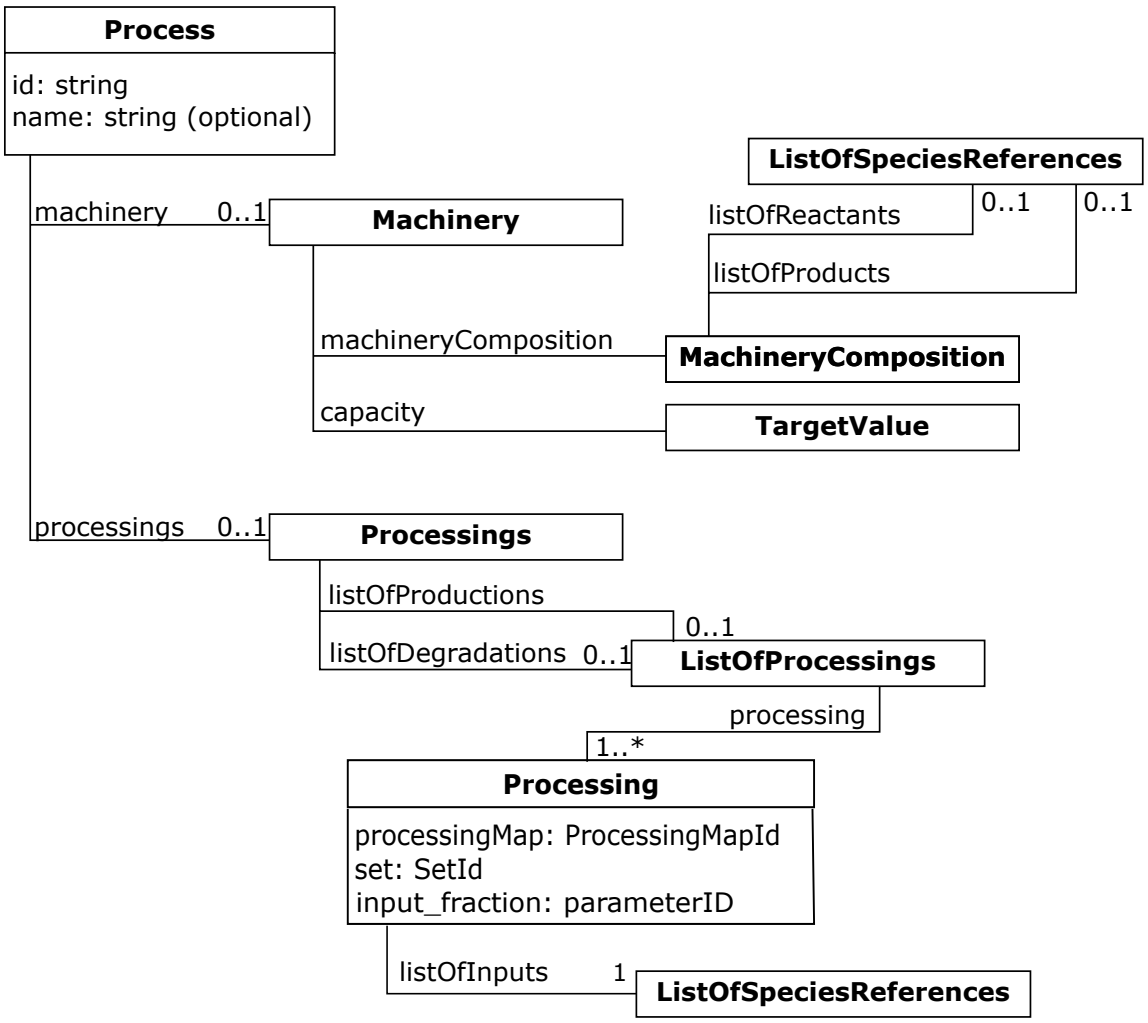

Figure 13: Class used to store processes.

Processes and **ListOfProcessingMaps**.

7.3 Process

The **Process** class is used to define cellular processes (Fig. 13).

A **Process** revolves around 2 optional substructures. The **Machinery** is the molecular entity enabling the process (*e.g.* ribosome for translation.) Each machinery unit has a limited production/degradation capacity. Every macromolecule produced by a process has a metabolite cost (metabolites needed to produce/degrade it and byproducts). However, if a machinery is defined, there is an additional cost to produce the machinery that will enable the production/degradation of the target. This is similar to the production of **Enzyme**s in order to catalyze metabolic **Reaction**s.

Processings define the sets of macromolecules that a process produces or degrades. The production reaction of a **Macromolecule** is determined by the **Process**es it goes

through. For example, a protein's production reaction is defined by listing the protein as an input in the **Processings** of the translation process. If a protein is not listed as an input of any process, its production reaction is empty, meaning that it does not cost anything to produce the protein. **Processings** break down **Macromolecule**s in metabolic **Species** and **Machinery** costs.

The *id* attribute The **id** attribute is a string defining the identifier of a process.

The *name* attribute The **name** attribute is a string that can be used to give the process a more human understandable name.

7.4 Machinery

The **Machinery** class defines the machinery used by a process (Fig. 13). **Machinery** has no simple attributes. If a **Machinery** is defined, it defines a *capacity constraint* (C_2) in RBA and the **Machinery** concentration is computed, otherwise it is not. Note that the processing costs are always taken into account in the mass balance constraint (C_1) in all cases (with ou without a **Machinery**). Every **Machinery** unit is produced according to the reaction defined by a **MachineryComposition**. Every unit also has a capacity defined by a **TargetValue**. The capacity defines how many targets a **Machinery** can process in 1 unit of time. Total capacity (base capacity multiplied by number of **Machinery** units) must always exceed the number of targets produced.

7.5 MachineryComposition

The **MachineryComposition** class defines the assembly of a complex molecular machinery (Fig. 13). **MachineryComposition** has no simple attributes. It contains two **ListOfSpeciesReferences**. One is for reactants, the other for potential byproducts of the assembly reaction of the complex itself (e.g. GDP when connecting ribosomal subunits). Note that in this case, **SpeciesReference**s can refer to *both* metabolic **Species** and **Macromolecule**s. The assembly reaction should contain obvious components of the machinery, but also metabolic costs related to assembly (such as ATP/GTP costs) *unless* these costs are already covered by a process.

7.6 Processings

The **Processings** relates **Macromolecule**s production/degradation to some given **Process**es (Fig. 13). **Processings** has no simple attributes. It may contain two **ListOfProcessings**, one for production and one for degradation.

7.7 Processing

The **Processing** class defines how **Macromolecule**s are produced/degraded (Fig. 13). **Processing** is used to break down **Macromolecule**s into metabolites by linking them

to a **ProcessingMap**. It contains one **ListOfSpeciesReferences** that lists **Macromolecules** that are inputs of this process. In this context, the species of a **SpeciesReference** must be a macromolecule. The stoichiometry attribute is used as an indicator, what fraction of all entities of the respective input species require this processing. If for example only 10% of a protein are folded by a given chaperone the stoichiometry for the protein as input species to the respective processing is 0.1. The default value for this argument is 1.0. Please note that this is a new functionality in *v3* and the stoichiometry attribute was ignored in previous versions (which is the equivalent of it being equal to 1.0).

The *processingMap* attribute The **processingMap** attribute must match the identifier of a **ProcessingMap**. This **ProcessingMap** will be used to compute the production/degradation reaction of **Macromolecules**, as well as **Machinery** costs.

The *set* attribute The **set** attribute must refer to a **Macromolecule** set. Currently, the only acceptable values are **protein**, **rna** and **dna**. **Macromolecules** that are listed as input must belong to this set.

The *input_fraction* attribute (optional) The **input_fraction** attribute must refer to a model parameter (**Function** or **Aggregate**). It specifies what proportion of the defined input species require the respective processing. If not specified *100%* is assumed. **PLEASE NOTE:** We suggest not to define the *input_fraction* attribute as a growth-rate dependent function (unless it is a deliberate modelling choice by the user); since this leads to a severe increase in computation time. Instead we suggest to define it as a function of an environ- mental variable defined in medium.tsv.

7.8 ProcessingMap

The **ProcessingMap** class is used to convert **Macromolecules** in metabolic and machinery costs (Fig.14).

There are two types of processings. The **ConstantProcessing** lists metabolites that are always consumed or produced when processing a macromolecule, no matter its composition (*e.g.* translation initiation). The **ListOfComponentProcessing** container details **ComponentProcessings** depending on the individual **Components** of the **Macromolecule**. They cover metabolites used to assemble the **Component** onto the nascent **Macromolecule**. They also cover machinery costs, *i.e.* how many **Machinery** units are needed to assemble the **Component**.

The *id* attribute The **id** attribute is a string defining the identifier of a processing map.

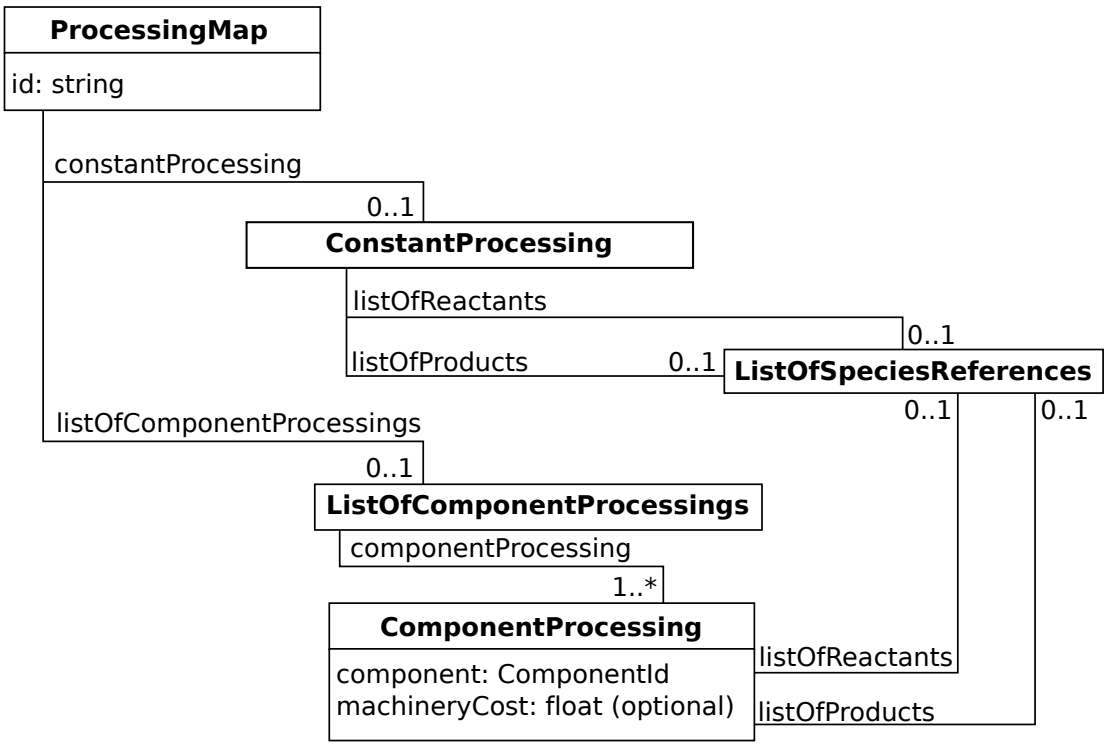

Figure 14: Class used to compute production/degradation of macromolecules.

7.9 ConstantProcessing

The **ConstantProcessing** class defines metabolites consumed and byproducts generated by an assembly process (Fig.14). It contains two **ListOfSpeciesReferences**, one for metabolites consumed and one for metabolites produced. Note that in this context, a **SpeciesReference** must refer to a metabolic **Species**.

7.10 ComponentProcessing

The **ComponentProcessing** class defines metabolites consumed and byproducts generated when assembling a specific **Component** (Fig.14). It contains two **ListOfSpeciesReferences**, one for metabolites consumed and one for metabolites produced. Note that in this context, a **SpeciesReference** must refer to a metabolic **Species**. Additionally, it defines a machinery cost used in a **Machinery**'s capacity constraint.

The *component* attribute The **component** attribute is a string that must match the identifier of a **Component**.

The *machineryCost* attribute The **machineryCost** attribute is a real value that is used to compute how many **Machinery** units are needed to assemble the **Component**. For example, let the machinery cost for the processing of an amino acid be 1. The

capacity of the **Machinery** (the ribosome) is the number of amino acids it can assemble per unit of time. The machinery cost allows to compute how many ribosomes are needed to produce the **Component** and, in the end, the **Macromolecule** (in this example the number of amino acids divided by the ribosome's capacity). Note that if a **Machinery** is not defined, the **machineryCost** must be set to 0.

7.11 Examples

The best example to illustrate processes is protein translation (Fig. 15). The XML structures are quite long, but processes depend on 3 relatively simple substructures. First, we define the process machine. The definition is reminiscent of enzymes: a machine composition, here all ribosome components (proteins and RNAs), and a catalytic rate, here expressed in number of amino acids per hour. Second, we list all macromolecules that undergo the process, here all proteins. Finally, the processing map, containing one reaction per macromolecule component. The question this structure answers is the following: my macromolecule contains a component named "alanine residue", how do I produce it from metabolites? Is the process machine involved? The answers are: for every alanine residue, you consume a charged tRNA, water and GTP, by-producing an uncharged tRNA, GDP, phosphate and protons. The machine cost is an additive cost that is related to the capacity defined earlier. For translation, we define the machine cost for every amino acid to be 1. In the end the total production cost of a protein is its number of amino acids. We defined the ribosome capacity in terms of amino acid per hour, say 1000. If a protein has 500 amino acids, a single ribosome will be able to produce two copies of this protein per hour.

The machine costs define a capacity constraint for the cell: in order to be viable, the cell needs not only to produce essential macrocomponents, but also the machines that assemble these macrocomponents. Production of machines cannot be neglected, as they increase very rapidly. When the production rate of macromolecule increases, the amount of machines needed for production increases in a seemingly linear fashion. However, machines are themselves composed of macromolecules (Fig. 16), and you need more machines to assemble those. Roughly speaking, if you want more proteins, you need to produce more ribosomes. But if you want more ribosomes, you need ribosomes to produce the new ribosomes. In the end the amount of machines increases more than linearly, imposing strong constraints on the cell.

```
<RBAProcesses>
  <listOfProcesses>
    <process id="P_TA" name="Translation">
      <machinery>
        <machineryComposition>
          <listOfReactants>
            <speciesReference species="P46899_loc_Cytoplasm" stoichiometry="1
            <speciesReference species="P26908_loc_Cytoplasm" stoichiometry="1
            <speciesReference species="rrnO-23S_loc_Cytoplasm" stoichiometry=
                ...
            <speciesReference species="M_gtp_c" stoichiometry="2.0"/>
            <speciesReference species="M_h2o_c" stoichiometry="2.0"/>
          </listOfReactants>
          <listOfProducts>
            <speciesReference species="M_gdp_c" stoichiometry="2.0"/>
            <speciesReference species="M_p_c" stoichiometry="2.0"/>
            <speciesReference species="M_h_c" stoichiometry="2.0"/>
          </listOfProducts>
        </machineryComposition>
        <capacity value="ribosome_capacity"/>
      </machinery>
      <processings>
        <listOfProductions>
          <processing processingMap="translation" set="protein">
            <listOfInputs>
              <speciesReference species="BSU29680_loc_Cytoplasm" stoichiometr
              <speciesReference species="BSU39060_loc_Cell_membrane" stoichio
                  ...
              <speciesReference species="P17631_loc_Cytoplasm" stoichiometry=
            </listOfInputs>
          </processing>
        </listOfProductions>
      </processings>
    </process>
      ...
  </listOfProcesses>
```

```
  <listOfProcessingMaps>
    <processingMap id="translation">
      <constantProcessing>
        <listOfReactants>
          <speciesReference species="M_tfmet_c" stoichiometry="1.0"/>
          <speciesReference species="M_gtp_c" stoichiometry="1.0"/>
          <speciesReference species="M_h2o_c" stoichiometry="2.0"/>
        </listOfReactants>
        <listOfProducts>
          <speciesReference species="M_trna_c" stoichiometry="1.0"/>
          <speciesReference species="M_fmet_c" stoichiometry="1.0"/>
          <speciesReference species="M_gdp_c" stoichiometry="1.0"/>
          <speciesReference species="M_p_c" stoichiometry="1.0"/>
          <speciesReference species="M_h_c" stoichiometry="1.0"/>
        </listOfProducts>
      </constantProcessing>
      <listOfComponentProcessings>
        <componentProcessing component="A" machineryCost="1.0">
          <listOfReactants>
            <speciesReference species="M_tala_c" stoichiometry="1.0"/>
            <speciesReference species="M_gtp_c" stoichiometry="2.0"/>
            <speciesReference species="M_h2o_c" stoichiometry="2.0"/>
          </listOfReactants>
        </componentProcessing>
            ...
        <componentProcessing component="CHEBI:24875" machineryCost="0.0">
          <listOfReactants>
            <speciesReference species="M_fe2_c" stoichiometry="1.0"/>
          </listOfReactants>
        </componentProcessing>
      </listOfComponentProcessings>
    </processingMap>
      ...
  </listOfProcessingMaps>
</RBAProcesses>
```

Figure 15: processes.xml from the hand curated model for model bacteria *B. subtilis*. Large chunks of the files were removed for brevity. This example focuses on some aspects of the definition of the translation process. The first element that is defined is the process machine: the ribosome. The machine is defined as a single assemble reaction from macromolecules (ribosomal proteins and rRNAs) and metabolites (GTP needed to assemble ribosomal proteins and rRNAs). Process machines have a catalytic rate termed capacity: for the ribosome, this is the number of amino acids processed per hour, later defined by the parameter P_TA_capacity. Finally the process contains the list of macromolecules to produce or degrade, here all the proteins in the model. A processing map explicits how macromolecules are produced from their components. Here, the processing map starts by defining metabolites that are consumed and produced independent of protein composition (translation initiation). We also show how an alanine residue is built from metabolites.

```xml
<RBAProcesses>
  <listOfProcesses>
    <process id="P_TA" name="Translation process">
      <machinery>
        <machineryComposition>
          <listOfReactants>
            <speciesReference species="small_protein_loc_Cytoplasm" stoichiometry="1.0"/>
            <speciesReference species="large_protein_loc_Cytoplasm" stoichiometry="1.0"/>
          </listOfReactants>
        </machineryComposition>
        <capacity value="ribosome_capacity"/>
      </machinery>
      <processings>
        <listOfProductions>
          <processing processingMap="translation" set="protein">
            <listOfInputs>
            <speciesReference species="small_protein_loc_Cytoplasm" stoichiometry="1.0"/>
            <speciesReference species="large_protein_loc_Cytoplasm" stoichiometry="1.0"/>
            </listOfInputs>
          </processing>
        </listOfProductions>
      </processings>
    </process>
        ...
  </listOfProcesses>
  <listOfProcessingMaps>
    <processingMap id="translation_map">
      <constantProcessing>
        <listOfReactants>
          <speciesReference species="M_carbon_source_c" stoichiometry="1.0"/>
        </listOfReactants>
      </constantProcessing>
      <listOfComponentProcessings>
        <componentProcessing component="protein_component_residue" machineryCost="1.0">
          <listOfReactants>
            <speciesReference species="M_protein_component_precursor_c" stoichiometry="1.0"/>
            <speciesReference species="M_carbon_source_c" stoichiometry="1.0"/>
          </listOfReactants>
        </componentProcessing>
      </listOfComponentProcessings>
    </processingMap>
  </listOfProcessingMaps>
</RBAProcesses>
```

Figure 16: processes.xml from the minimal model. We only define one process corresponding to pseudo-translation. It contains a two-protein machine with a capacity that we will define later. We list all the proteins from the model as input of translation. Proteins only had one component so the processing map is short. We define a constant cost corresponding to translation initiation and metabolites consumed for every residue that has to be assembled.

8 targets.xml

The targets file is used to define production and degradation constraints, i.e. fluxes of metabolic **Species** or **Macromolecule**s that must be maintained for the cell to be functional.

8.1 Rationale

In an RBA model, the production of machines (enzymes and process machines) is automatically computed to sustain the metabolic fluxes and the production of macromolecules necessary to achieve optimal growth rate. However, in order to grow, the cell must produce more than just enzymes and ribosomes. These production constraints are defined in targets.xml as fluxes or concentrations that a cell must maintain in order to be functional.

Typical targets include molecules such as DNA, mRNAs, or housekeeping proteins whose concentration must be maintained at all times. Structural molecules contained in cell wall for example, must also be maintained at a constant concentration. A less obvious example are essential metabolites whose concentration is high and well regulated, such as NADP/NADPH. Defining a target concentration for NADP forces de novo production of NADP, otherwise the model assumes that recycling existing NADP is sufficient to achieve any growth rate. We suggest defining target concentrations for metabolites whenever metabolomic data are available.

In general, targets are defined as concentrations to maintain, but it's also possible to specify absolute fluxes of production for metabolites or reactions. A typical reaction flux that must be maintained is the production of maintenance ATP (ATP used by secondary processes of the cell).

From the XML point of view, the constraints may be equalities (the concentration must be maintained to that exact value) or inequalities (the concentration must be maintained between these bounds).

8.2 RBATargets

The outermost part of the process file is an instance of class **RBATargets**, shown in Figure 17.

RBATargets has no simple attributes. It contains 3 **ListOfTargetSpecies**. These targets allow to define metabolic **Species** or **Macromolecule** fluxes. One is for production fluxes, another for degradation fluxes. The last list is for maintaining a target at a given concentration. The difference with a simple production flux is that keeping a target at a concentration depends on growth rate. More precisely, the flux needed to keep the concentration is the growth rate multiplied by the target concentration. Note that all fluxes must be positive. If the target is a **Macromolecule**, production/degradation can only occur if the **Processings** section of some **Process** defines how the **Macromolecule** is actually produced/degraded.

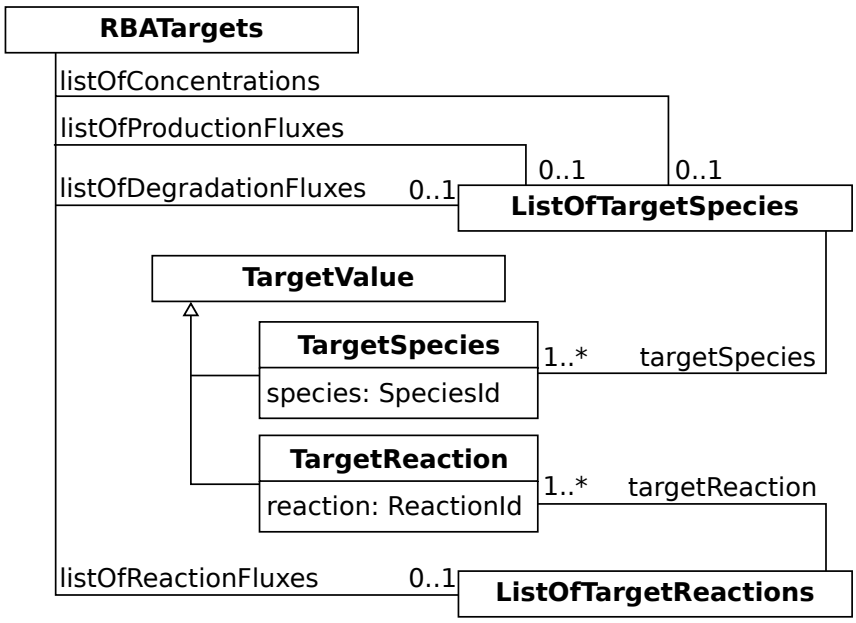

Figure 17: XML structure of target document.

It contains a **ListOfTargetReactions**. It is also possible to define target fluxes as reaction fluxes. These targets add constraints on the flux of a specific metabolic **Reaction**. In this case, fluxes may be positive or negative.

8.3 TargetSpecies

The **TargetSpecies** class defines constraints for a species flux (Fig. 17). It inherits **TargetValue** for the constraint definition part, allowing for equality or inequality constraints.

The *species* attribute The **species** attribute is a string that must match the identifier of a metabolic **Species** or a **Macromolecule**. Note that the **Macromolecule** must be broken down into metabolite costs through the **Processings** section of some **Process**. Otherwise no cost will be applied.

8.4 TargetReaction

The **TargetReaction** class defines constraints for a reaction flux (Fig. 17). It inherits **TargetValue** for the constraint definition part, allowing for equality or inequality constraints.

The *reaction* attribute The **reaction** attribute is a string that must match the identifier of a metabolic **Reaction**.

8.5 Examples

Most targets in an RBA model are concentration targets for DNA production, mRNA production, housekeeping protein production, de novo metabolite production, by specifying a concentration that must be maintained (Fig. 18 and 19).

The second class of targets are fluxes related to degradation. For example, our *B. subtilis* model explicitly accounts for mRNA degradation. As a result, there are 3 target fluxes associated with mRNAs: a flux associated with the target concentration to maintain, a flux indicating how many molecules are degraded by seconds (how mRNAs are degraded is defined in processes.xml), and an absolute production flux that exactly compensates degradation (Fig. 18).

The third class of targets are targets that reference reactions instead of metabolic species or macromolecules. Maintenance ATP is one of the most common examples, but there are a lot of contextual requirements that can be written in this form, such as flagella movement, response to stresses such as oxidative stresses causing NADPH consumption.

```xml
<RBATargets>
  <listOfTargetGroups>
    <targetGroup id="translation_targets">
      <listOfConcentrations>
        <targetSpecies value="nonenzymatic_proteins_Secreted" species="average_protein_loc_Secreted"/>
        <targetSpecies value="nonenzymatic_proteins_Cell_membrane" species="average_protein_loc_Cell_membrane"/>
        <targetSpecies value="nonenzymatic_proteins_Cytoplasm" species="average_protein_loc_Cytoplasm"/>
      </listOfConcentrations>
    </targetGroup>
    <targetGroup id="transcription_targets">
      <listOfConcentrations>
        <targetSpecies value="mrna_concentration" species="mrna_loc_Cytoplasm"/>
      </listOfConcentrations>
      <listOfProductionFluxes>
        <targetSpecies value="mrna_degradation_flux" species="mrna_loc_Cytoplasm"/>
      </listOfProductionFluxes>
    </targetGroup>
    <targetGroup id="replication_targets">
      <listOfConcentrations>
        <targetSpecies value="dna_concentration" species="dna_loc_Cytoplasm"/>
      </listOfConcentrations>
    </targetGroup>
    <targetGroup id="rna_degradation">
      <listOfDegradationFluxes>
        <targetSpecies value="mrna_degradation_flux" species="mrna_loc_Cytoplasm"/>
      </listOfDegradationFluxes>
    </targetGroup>
        ...
    <targetGroup id="maintenance_atp_target">
      <listOfReactionFluxes>
        <targetReaction lowerBound="maintenance_atp" reaction="R_maintenance_atp"/>
      </listOfReactionFluxes>
    </targetGroup>
    <targetGroup id="flagella_activation">
      <listOfReactionFluxes>
        <targetReaction value="flagella_proton_flux" reaction="Th"/>
      </listOfReactionFluxes>
    </targetGroup>
  </listOfTargetGroups>
</RBATargets>
```

Figure 18: targets.xml from a hand-curated model for the model bacteria *B. subtilis*. Large parts of the files where removed for brevity. The first 3 targetSpecies encompass all nonenzymatic proteins that must be produced by the cell for housekeeping purposes. mRNAs have two production constraints: the first contraint defines a flux that must be generated in order to maintain their concentration, the second constraint defines a flux that must be generated in order to compensate degradation (defined later in the file). Finally, there are two example of reaction targets: a lowerBound for maintainance ATP production, and a set bound for the reaction controlling flagella movement.

```
<RBATargets>
  <listOfTargetGroups>
    <targetGroup id="biomass_production">
      <listOfConcentrations>
        <targetSpecies value="target_biomass_production" species="M_biomass_c"/>
    </targetGroup>
  </listOfTargetGroups>
</RBATargets>
```

Figure 19: targets.xml from the minimal model. We pool all non-machine production requirements in a generic biomass production requirement.

9 parameters.xml

The parameter file contains user-defined parameters and functions.

9.1 Rationale

The file parameters.xml contains all numerical values occurring in the model (except for stoichiometries). Nearly all other files refer to this file, as we have seen throughout the examples, where numerical values were defined as parameter identifiers. The most common parameters are: total amino acid concentrations, fractions of protein per compartment, percentages of non-enzymatic protein per compartment and cellular machinery, target fluxes for metabolites and macromolecules, efficiencies of enzymes, transporters and molecular machines.

A parameter may be defined as: a constant, a function of growth rate, of an environmental variable (defined in medium.tsv) or an aggregate of several functions. Currently, the format supports the following function types: constant, linear, quadratic, inverse, exponential,logistic, Hill/Michaelis-Menten with and without competetive inhibion and the Arrhenius function, as well as its centered variant. The function types have been chosen to reflect common biochemical functions. Typically, enzyme efficiencies vary linearly with growth rate and transporters have activities that depend on the concentration of metabolites transported according to a Michaelis-Menten function.

A parameter can also be defined as a product or sum of functions, called an "aggregate". For example, the activity of a transporter can be described as the product of a growth rate-dependent maximal activity and a concentration-dependent Michaelis-Menten term. Please note that complex parameters, depending on multiple independent variables can be represented by nesting aggregates. All functions are allowed to have growth-rate or concentration dependencies. For instance, maximal densities and target fluxes can be defined as constant or growth rate dependent.

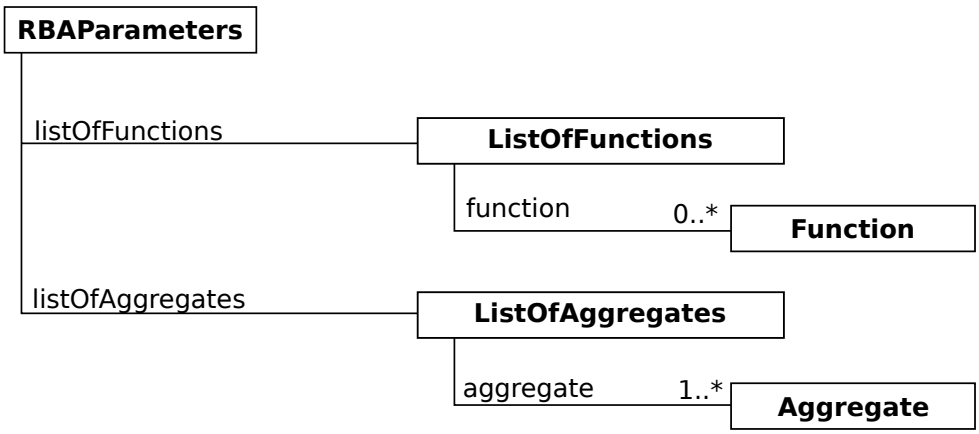

Figure 20: XML structure of parameter document.

9.2 RBAParameters

The outermost part of the parameter file is an instance of class **RBAParameters**, shown in Figure 20.

RBAParameters has no simple attributes. It includes exactly one instance of **ListOf** container classes. All **ListOf** classes do not have own attributes, they are merely used to organize a list of instances from another class.

9.3 Function

The **Function** class is used for user-defined functions and parameters (Fig. 21). The default variable of a function is the growth rate, but it may also be the extracellular concentration of a metabolite. Every function holds a **ListOfParameters**, where **Parameter** are defined according to each type of function.

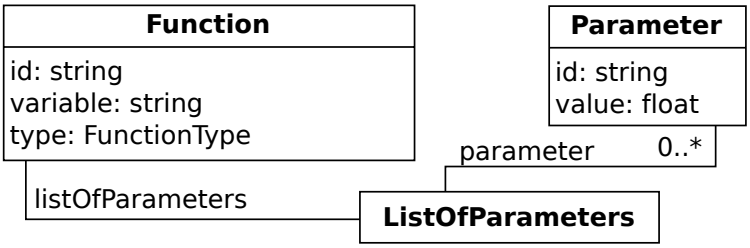

Figure 21: Class used to store user-defined functions.

The *id* attribute The **id** attribute is a string defining the identifier of the function.

The *variable* attribute The **variable** attribute is a string defining the variable of the function. If empty or set to `growth_rate`, the variable is the current growth rate. Alternatively, the variable may the prefix of a metabolite.

The *type* attribute The **type** attribute is a string that must match a known function type. Currently, the supported types are:

- **constant**. Constant function with parameter *CONSTANT*.
- **linear**. Linear function ($f(x) = ax + b$)
  with parameters *LINEAR_ CONSTANT* (b), *LINEAR_ COEF* (a), *X_ MIN*, *X_ MAX*, *Y_ MIN*, *Y_ MAX*.
  The 4 last parameters are used to saturate the function. The computation is done in three steps. First, if the variable (e.g. growth rate) is outside of the [X_MIN, X_MAX] range, it is set to the closest value in that range. Second, the function is computed. Finally, if the return value is outside of the [Y_MIN, Y_MAX] range, it is set to the closest value in that range. The range parameters can be set to infinity by setting them to `("-inf")` or `("inf")`.
- **quadratic**. Quadratic function ($f(x) = ax^2 + bx + c$)
  with parameters *QUADRATIC_ TERM_ COEF* (a), *LINEAR_ TERM_ COEF* (b),*CONSTANT* (c), *X_ MIN*, *X_ MAX*, *Y_ MIN*, *Y_ MAX*.
  This polynomial is computed like the linear function.
- **inverse**. Inverse function ($f(x) = \frac{x_{max}}{x}$) with parameters *X_ MAX*.
- **exponential**. Exponential function ($f(x) = A\exp(\lambda x + c)$) with parameters *RATE* (λ) and *CONSTANT* (c) (optional with default value of 0.0) and MULTIPLIER (A) (optional with default value of 1.0).
- **indicator**. Indicator function with parameters *X_ MIN* and *X_ MAX*. This function returns one if the variable (growth rate) is in the interval [X_MIN, X_MAX], zero otherwise.
- **logistic**. Logistic function ($f(x) = \frac{A}{B+(C*\exp(-D*x)}$) with parameters A,B,C and D.
  This function returns one if the variable (growth rate) is in the interval [X_MIN, X_MAX], zero otherwise.
- **michaelisMenten**. Irreversible Michaelis Menten ($f(x) = k_{max}\frac{x^n}{x^n+K_M^n}$) function with parameters *kmax*, *Km*, *HILL_ COEFFICIENT* (n, optional with default value of 1) and *Y_ MIN* (optional). If Y_MIN is defined, any return value lower than Y_MIN will be set to Y_MIN.
- **competitiveInhibition**. Michaelis Menten function with competetive inhibion ($f(x) = k_{max}\frac{x}{x+K_M(1+\frac{I}{K_I})}$) with parameters *kmax*, *Km*, *Ki*, *I* and *Y_ MIN* (optional). Please note that the inhibitor concentration I is a function parameter and not an independent variable If Y_MIN is defined, any return value lower than Y_MIN will be set to Y_MIN.

- **arrhenius**. Arrhenius function ($f(x) = A\exp(-\frac{E_A}{R*x})$) with parameters *PRE_EXPONENTIAL_FACTOR* (A), *ACTIVATION_ENERGY* (E_A), *GAS_CONSTANT* (R). If the return value is outside of the [Y_MIN, Y_MAX] range, it is set to the closest value in that range.

- **centeredArrhenius**. Centered Arrhenius function to a reference temperature ($f(x) = A\exp(-(\frac{1}{x} - \frac{1}{T_r}) * \frac{E_A}{R})$) with parameters *REFERENCE_TEMPERATURE* (T_r), *PRE_EXPONENTIAL_FACTOR* (A), *ACTIVATION_ENERGY* (E_A), *GAS_CONSTANT* (R). If the return value is outside of the [Y_MIN, Y_MAX] range, it is set to the closest value in that range.

9.4 Parameter

The **Parameter** class is used to store the values of function parameters (Fig. 21).

The *id* attribute The **id** attribute is a string that should match a valid parameter identifier. The list of valid parameters for each type of **Function** is listed above.

The *value* attribute The **value** attribute is a real number representing the value of the attribute.

9.5 Aggregate

The **Aggregate** class is used to assemble user-defined functions (Fig. 22). Every aggregate holds a **ListOfFunctionReferences** and **ListOfAggregateReferences**, where each **FunctionReference** and **AggregateReference** refers to a previously defined function or aggregate.

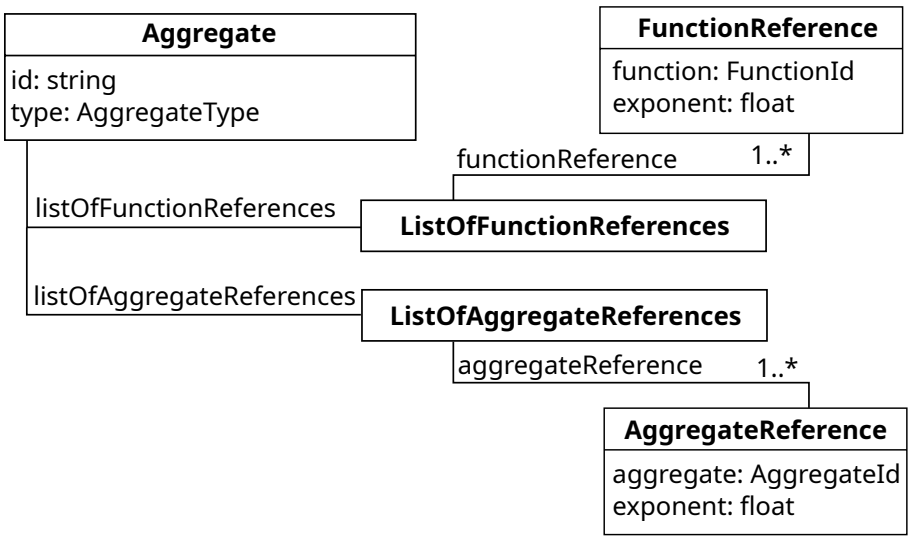

Figure 22: Class used to store user-defined aggregates.

The *id* attribute The **id** attribute is a string defining the identifier of the aggregate.

The *type* attribute The **type** attribute is a string that must match a known aggregate type. Currently, the supported types are:

- **multiplication**. The result is the multiplication of the current values returned by the functions and aggregates referenced in the aggregate.
- **addition**. The result is the sum of the current values returned by the functions and aggregates referenced in the aggregate.

9.6 FunctionReference

The **FunctionReference** class is used to refer to a user-defined **Function** (Fig. 22).

The *function* attribute The **function** attribute is a string that must match the identifier of a user-defined **Function**.

The *exponent* attribute The **exponent** attribute represents the exponent of the respective function value in the aggregate. (Division can for example be represented by formulating a multiplicative aggregate, where the function value representing the denominator has the exponent -1)

9.7 AggregateReference

The **AggregateReference** class is used to refer to another user-defined **Aggregate** (Fig. 22).

The *aggregate* attribute The **aggregate** attribute is a string that must match the identifier of a user-defined **Aggregate**.

The *exponent* attribute The **exponent** attribute represents the exponent of the respective aggregate value in the aggregate. (Same as in **FunctionReference**)

9.8 Examples

parameters.xml is one of the longest files in the model as it contains all numerical values in the model. All parameters follow the same rules, no matter whether they define a density constraint, a target concentration or a catalytic activity.

In the first example (Fig. 23), we show the definition of a parameter related to the density constraint, protein_concentration. This parameter reflects measured protein densities, later used to define a growth-dependent maximal density bound. The measures showed that the density of proteins decreases with growth rate: the density bound becomes smaller for larger growth rates. We also show the definition of a transporter catalytic rate. A transporter can be defined in two parts: its base efficiency (potentially growth-rate dependent, here modeled as a constant), and transport factors related to the concentration of the transported molecule and potential cofactors.

```xml
<RBAParameters>
  <listOfFunctions>
    <function id="amino_acid_concentration" type="linear" variable="growth_rate">
      <listOfParameters>
        <parameter id="LINEAR_CONSTANT" value="6.3138"/>
        <parameter id="Y_MAX" value="inf"/>
        <parameter id="X_MAX" value="1.6"/>
        <parameter id="LINEAR_COEF" value="-0.9757"/>
        <parameter id="Y_MIN" value="-inf"/>
        <parameter id="X_MIN" value="0.25"/>
      </listOfParameters>
    </function>
        ...
    <function id="TptsG_enzyme_forward_efficiency" type="constant" variable="growth_rate">
      <listOfParameters>
        <parameter id="CONSTANT" value="1334026.4422"/>
      </listOfParameters>
    </function>
    <function id="TptsG_M_glc_e_transport_factor" type="michaelisMenten" variable="M_glc_e">
      <listOfParameters>
        <parameter id="Km" value="0.8"/>
        <parameter id="kmax" value="1.0"/>
        <parameter id="HILL_COEFFICIENT" value="1.0"/>
      </listOfParameters>
    </function>
        ...
  </listOfFunctions>
  <listOfAggregates>
        ...
    <aggregate id="TptsG_efficiency" type="multiplication">
      <listOfFunctionReferences>
        <functionReference function="TptsG_enzyme_forward_efficiency" exponent="1.0"/>
        <functionReference function="TptsG_M_glc_e_transport_factor" exponent="1.0"/>
      </listOfFunctionReferences>
    </aggregate>
        ...
  </listOfAggregates>
</RBAParameters>
```

Figure 23: parameters.xml from a hand-curated model for model bacteria *B. subtilis*. Large parts of the file were removed for brevity. Parameters are defined either as constants, functions (e.g. linear or Michealis-Menten), or aggregates, representing the multiplication of functions. Functions can be defined in terms of growth rate (the default) or concentration of external metabolites. Aggregates may contain an arbitrary number of functions, and functions in an aggregate can be defined according to different variables (growth rate, concentration of different metabolites).

In the example from the minimal model (Fig. 24), we see how quickly parameters accumulate: we need parameters for catalytic activities, process machines, target concentrations and density constraints.

```xml
<RBAParameters>
  <listOfFunctions>
    <function id="kcat_transport_base" type="constant" variable="growth_rate">
      <listOfParameters>
        <parameter id="CONSTANT" value="10"/>
      </listOfParameters>
    </function>
    <function id="kcat_precursor" type="constant" variable="growth_rate">
      <listOfParameters>
        <parameter id="CONSTANT" value="10"/>
      </listOfParameters>
    </function>
    <function id="kcat_biomass" type="constant" variable="growth_rate">
      <listOfParameters>
        <parameter id="CONSTANT" value="10"/>
      </listOfParameters>
    </function>
    <function id="zero" type="constant" variable="growth_rate">
      <listOfParameters>
        <parameter id="CONSTANT" value="0"/>
      </listOfParameters>
    </function>
    <function id="transport_factor" type="michaelisMenten" variable="M_carbon_source_e">
      <listOfParameters>
        <parameter id="Km" value="0.5"/>
        <parameter id="kmax" value="1.0"/>
        <parameter id="HILL_COEFFICIENT" value="1.0"/>
      </listOfParameters>
    </function>
    <function id="maximal_cytosol_density" type="constant" variable="growth_rate">
      <listOfParameters>
        <parameter id="CONSTANT" value="1"/>
      </listOfParameters>
    </function>
    <function id="target_biomass_production" type="constant" variable="growth_rate">
      <listOfParameters>
        <parameter id="CONSTANT" value="1"/>
      </listOfParameters>
    </function>
    <function id="ribosome_capacity" type="constant" variable="growth_rate">
      <listOfParameters>
        <parameter id="CONSTANT" value="100"/>
      </listOfParameters>
    </function>
  </listOfFunctions>
  <listOfAggregates>
    <aggregate id="kcat_transport" type="multiplication">
      <listOfFunctionReferences>
        <functionReference function="kcat_transport_base" exponent="1.0"/>
        <functionReference function="transport_factor" exponent="1.0"/>
      </listOfFunctionReferences>
    </aggregate>
  </listOfAggregates>
</RBAParameters>
```

Figure 24: parameters.xml from the minimal model. For simplicity, all parameters were defined as constants, except for transport terms, which adopt the traditional Michaelis-Menten.

Examples of other functions (quadratic, Arrhenius ...) are shown in Fig. 25. Several types of aggregates are also introduced. Such as multiplying functions, dividing one function by the other, adding functions together; as well as nesting aggregates aggregating aggregates (and functions).

```xml
<RBAParameters>
  <listOfFunctions>
    <function id="logistic_dummy" type="logistic" variable="growth_rate">
      <listOfParameters>
        <parameter id="A" value="1.0"/>
        <parameter id="B" value="2.0"/>
        <parameter id="C" value="3.0"/>
        <parameter id="D" value="4.0"/>
      </listOfParameters>
    </function>
    <function id="arrhenius_dummy" type="arrhenius" variable="Temperature">
      <listOfParameters>
        <parameter id="Y_MIN" value="0.0"/>
        <parameter id="Y_MAX" value="inf"/>
        <parameter id="ACTIVATION_ENERGY" value="2432.0"/>
        <parameter id="PRE_EXPONENTIAL_FACTOR" value="2.71"/>
        <parameter id="GAS_CONSTANT" value="8.3"/>
      </listOfParameters>
    </function>
    <function id="centered_arrhenius_dummy" type="centeredArrhenius" variable="Temperature">
      <listOfParameters>
        <parameter id="Y_MIN" value="0.0"/>
        <parameter id="Y_MAX" value="inf"/>
        <parameter id="ACTIVATION_ENERGY" value="2432.0"/>
        <parameter id="PRE_EXPONENTIAL_FACTOR" value="2.71"/>
        <parameter id="GAS_CONSTANT" value="8.3"/>
        <parameter id="REFERENCE_TEMPERATURE" value="37.0"/>
      </listOfParameters>
    </function>
    <function id="ribosome_efficiency_MM" type="michaelisMenten" variable="growth_rate">
      <listOfParameters>
        <parameter id="kmax" value="864000"/>
        <parameter id="Km" value="0.5"/>
        <parameter id="Y_MIN" value="86400"/>
        <parameter id="HILL_COEFFICIENT" value="1.0"/>
      </listOfParameters>
    </function>
    <function id="fraction_active_ribosomes" type="exponential" variable="growth_rate">
      <listOfParameters>
        <parameter id="RATE" value="-0.083333"/>
        <parameter id="CONSTANT" value="0.0"/>
      </listOfParameters>
    </function>
    <function id="dummy_quadratic" type="quadratic" variable="growth_rate">
      <listOfParameters>
        <parameter id="QUADRATIC_TERM_COEF" value="3.0"/>
        <parameter id="LINEAR_TERM_COEF" value="2.0"/>
        <parameter id="CONSTANT" value="-1.0"/>
        <parameter id="X_MAX" value="inf"/>
        <parameter id="X_MIN" value="0.0"/>
      </listOfParameters>
    </function>
  </listOfFunctions>
```

```xml
    <listOfAggregates>
      <aggregate id="Addition_aggregate" type="addition">
        <listOfFunctionReferences>
          <functionReference function="dummy_constant_10" exponent="1.0"/>
          <functionReference function="dummy_constant_10" exponent="1.0"/>
        </listOfFunctionReferences>
      </aggregate>
      <aggregate id="Division_aggregate" type="multiplication">
        <listOfFunctionReferences>
          <functionReference function="dummy_quadratic" exponent="1.0"/>
          <functionReference function="dummy_constant_10" exponent="-1.0"/>
        </listOfFunctionReferences>
      </aggregate>
      <aggregate id="Nested_aggregate_1" type="multiplication">
        <listOfAggregateReferences>
          <functionReference function="Addition_aggregate" exponent="1.0"/>
          <functionReference function="Division_aggregate" exponent="1.0"/>
        </listOfAggregateReferences>
      </aggregate>
      <aggregate id="Nested_aggregate_2" type="addition">
        <listOfAggregateReferences>
          <functionReference function="Nested_aggregate_1" exponent="1.0"/>
        </listOfAggregateReferences>
        <listOfFunctionReferences>
          <functionReference function="dummy_constant_10" exponent="1.0"/>
        </listOfFunctionReferences>
      </aggregate>
    </listOfAggregates>
  </RBAParameters>
```

Figure 25: XML format of user-defined functions and aggregates.

10 custom_constraints.xml

The custom_constraints file removes/disables native constraints of the *RBA*-problem and defines custom linear constraint between one or more decision variables and one or more model parameters (see *eq.* 1). Constraints can be defined as inequality or equality and refer to *RBA*-problem variables (weighted by constant coefficients, model parameters and potentially proportional to growth rate, see *eq.* 2), model parameters (weighted by constant coefficients and potentially proportional to growth rate, see *eq.* 3).

$$\sum_{i \in V_{refs}} (\alpha_i x_i) \; + \sum_{j \in P_{refs}} (\beta_j p_j) \; \lesseqgtr \; b \tag{1}$$

Weights of decision variables (noted x_i) are defined in class **VariableReference**, as the product of a constant (c_i), a model parameter (p_i) and the growth rate (μ). Where the latter two are optional.

$$\alpha_i = c_i \; p_i \;\; or \;\; c_i \; p_i \; \mu \tag{2}$$

Weights of model parameters are defined in class **ParameterReference**, as the product of a constant (c_j) and optionally the growth rate (μ).

$$\beta_j = c_j \;\; or \;\; c_j \; \mu \tag{3}$$

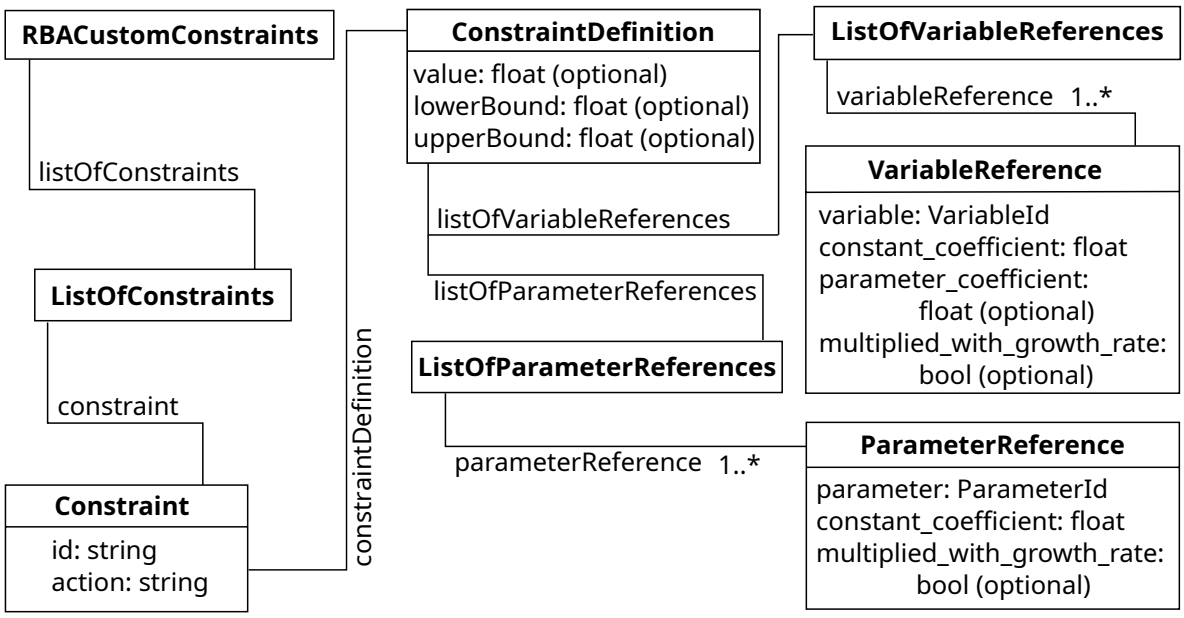

Figure 26: XML structure of custom_constraints document.

10.1 RBACustomConstraints

The outmost part of the custom_constraints xml structure is an instance of the class **RBACustomConstraints**. This class has no simple attributes and only holds instance of the **ListOf** container class (holding instances of the class **Constraint**). All **ListOf** classes do not have own attributes, they are merely used to organize a list of instances from another class.

10.2 Constraint

The **Constraint** class is used to change to *RBA*-problem constraints. Note that changing the *RBA*-problem can also be done using the package RBAtools[1]. The file custom_contraints.xml is expected to contain the permanent constraints that should be always integrated or removed in the RBA-model of your favorite organism. It holds two attributes (**id** and **action**). If a new constraint is added (i.e. **action** is set to *add*), it must contain the **ConstraintDefinition** class with the necessary information.

The *id* attribute The **id** attribute is a string representing the name of the constraint. If a new constraint is defined this id should not be one of the already existing native or custom *RBA*-problem constraints.

The *action* attribute The **action** attribute is a string (*add*, *remove* or *ignore*).

An existing constraint can be removed by setting **action** to *remove* and **id** to a constraint identifier used in the *RBA*-problem. A new constraint can be defined by setting **action** to *add* and **id** to a unique identifier absent from the *RBA*-problem. (This requires the necessary information in **ConstraintDefinition**)
The **action** attribute can be set to *ignore* if the user wants to define constraints in the file, but without consequences. E.g. for testwise "switching" constraints on and off.

10.3 ConstraintDefinition

This class holds all necessary information to define a constraint, according to *eq.* 1.

The *value*, *lowerBound* and *upperBound* attributes Every attribute can be left undefined, or contain a float (b in *eq.* 1). If **value** is defined, the constraint is an equality constraint. Attributes **lowerBound** (resp. **upperBound**) define a lower bound (resp. upper bound) inequality constraint.

The lefthand side of the constraint is defined by lists of instances of classes **VariableReference** and **ParameterReference**, representing the elements in the respective sums in *eq.* 1.

[1] https://rba.inrae.fr/rbatools.html

10.4 VariableReference

This class represents each referenced decision variable (x_i) and defines the respective coefficient (α_i) in the constraint (see *eq.* 2).

The *variable* attribute The **variable** attribute is a string corresponding to the id of the decision variable/column in the *RBA*-problem.

The *parameter_coefficient* attribute The **parameter_coefficient** attribute is a string corresponding to the identifier of a model parameter (**Function** or an **Aggregate**).

The *constant_coefficient* attribute The **constant_coefficient** attribute is a float.

The *multiplied_with_growth_rate* attribute The **multiplied_with_growth_rate** attribute is a boolean. It indicates whether the respective coefficient in the *RBA*-problem is proportional (multiplied with) to the growth rate.

10.5 ParameterReference

This class represents each referenced model parameter (p_i) and defines the respective coefficient (β_i) in the constraint (see *eq.* 3).

The *parameter* attribute The **parameter** attribute is a string corresponding to the id of the model parameter (**Function** or an **Aggregate**).

The *constant_coefficient* attribute The **constant_coefficient** attribute is a float.

The *multiplied_with_growth_rate* attribute The **multiplied_with_growth_rate** attribute is a boolean. It indicates whether the respective coefficient in the *RBA*-problem is proportional (multiplied with) to the growth rate.

10.6 Examples

Examples of user-defined constraints in the custom_constraints file are given in Fig. 27. The original capacity constraint of the glucose transporter is removed and a new one, which imposes a given amount (specified by parameter) of unused reserve transporter is added. Two variants of coupling (imposing a given flux ratio) two metabolic reactions in the model (one with a fixed ratio and one with a ratio, proportional to the growth rate) are defined in the file, but ignored (not imposed).

```xml
<RbaCustomConstraints>
  <listOfConstraints>
    <Constraint id="R_glctransport_enzyme_forward_capacity" action="remove"/>
    <Constraint id="R_glctransport_enzyme_forward_capacity_new" action="add">
      <ConstraintDefinition upperBound="0.0">
        <listOfVariableReferences>
          <variableReference variable="R_glctransport" constant_coefficient="1.0"/>
          <variableReference variable="R_glctransport_enzyme" constant_coefficient="-1.0" parameter_coefficient="glctransport_efficiency"/>
        </listOfVariableReferences>
        <listOfParameterReferences>
          <parameterReference parameter="reserve_transporter_amount" constant_coefficient="1.0" multiplied_with_growth_rate="false"/>
        </listOfParameterReferences>
      </ConstraintDefinition>
    </Constraint>
    <Constraint id="Coupling_fermentation_constraint_1" action="ignore">
      <ConstraintDefinition value="0.0">
        <listOfVariableReferences>
          <variableReference variable="R_fermentation" constant_coefficient="1.0" multiplied_with_growth_rate="false"/>
          <variableReference variable="R_glctransport" constant_coefficient="-0.7" multiplied_with_growth_rate="false"/>
        </listOfVariableReferences>
      </ConstraintDefinition>
    </Constraint>
    <Constraint id="Coupling_fermentation_constraint_2" action="ignore">
      <ConstraintDefinition value="0.0">
        <listOfVariableReferences>
          <variableReference variable="R_fermentation" constant_coefficient="1.0" multiplied_with_growth_rate="false"/>
          <variableReference variable="R_glctransport" constant_coefficient="-3.0" multiplied_with_growth_rate="true"/>
        </listOfVariableReferences>
      </ConstraintDefinition>
    </Constraint>
  </listOfConstraints>
</RbaCustomConstraints>
```

Figure 27: Example of custom_constraints.xml file.

11 Mean model

RBApy creates models holding the full combinatory of isoenzymes for one metabolic reaction. It automatically creates copies of the metabolic reaction to a one to one association between a metabolic reaction and its enzymes. In other words, for one reaction catalyzed by n enzymatic complexes, RBApy will create n metabolic reactions, one for each enzymatic complex. Therefore the model size can quickly increase, especially for polyploid organisms that could have multiple isoenzymes per reaction, and might cause numerical instabilities due to singularities in the optimization problem.

To overcome this situation, and because simulating the full complexity of the model could not be informative, RBApy also creates two files called mean_composition_model_enzymes.xml and mean_composition_model_metabolism.xml, where all duplicated reactions have been removed.

Let be a possible enzymatic complexes composed of two subunits A and B: A with a stoichiometry s_A and being encoded by two genes g_A^1 and g_A^2, B with a stoichiometry s_B, and beging encoded by a single gene g_B^1. For the mean model, RBApy will create an **Enzyme** with a **MachineryComposition** composed of two **SpeciesReference** for g_A^1, g_A^2 having a stoichiometry of $s_A/2$ and one **SpeciesReference** for g_B^1 having a stoichiometry of s_B.

The choice for simulating the mean model instead on the complex one is made in model_file_index.in by setting the variables **metabolism** and **enzymes** to mean model files. Note that the file custom_contraint.xml might be updated also.